

Endangered Orbits

Game Technical Report

A Real-Scale Space Simulation: Orbital Mechanics,
GPU Atmospheric Scattering, and Rigid-Body Spacecraft Physics in Unity



Jacob Tang

(Tang Zi Jian)

Developed in the Unity Engine

Abstract

Endangered Orbits is a space-simulation game built in the Unity engine with the explicit goal of bridging a computer-science and an information-technology perspective on game development through the mathematics and physics that govern how objects move and rotate in a scene. Its foremost emphasis is on realistic physics at real cosmic scale, giving it greater depth than comparable titles in spacecraft movement physics and orbital mechanics. Its second emphasis is graphical fidelity, achieved through mathematical compute shaders that run natively on the GPU as a post-processing technique conceptually similar to ray tracing. This report documents the engineering behind four pillars of the project: (i) a **planetarium module** that solves Keplerian two-body and N -body orbital mechanics with high-order Runge–Kutta integrators; (ii) a physically based **atmospheric-scattering** shader implementing Rayleigh and Mie scattering per pixel; (iii) a **ship module** that reconstructs Unity’s rigid-body system externally to compute composite centres of mass and inertia tensors for modular spacecraft; and (iv) the **modular construction and pathfinding** systems that let players build ships in flight and let NPCs navigate empty space.

Contents

Abstract	1
1 Introduction	4
2 Overview of the Implementation	5
2.1 Scale of the Planetary System	5
2.2 Scale of Objects	5
2.3 Realistic Behaviour of the Spaceship	6
2.4 Control of the Spaceship	7
2.4.1 Modular Ship Construction	7
2.4.2 Pathfinding	8
2.4.3 Object Avoidance	8
2.4.4 Applying Movement	9
3 Math and Physics: Planetarium Module	9
3.1 Keplerian Component: Time	9
3.2 Keplerian Component: Reference Frame	10
3.3 Keplerian Component: Orbital Elements to State Vector	11
3.4 State Vector to Orbital Elements	19
3.5 N-Body Integrator	22
3.6 Numerical Runge–Kutta Methods	23
3.6.1 RK4 Scheme	23
3.6.2 RK8 Scheme	24
3.6.3 Heliocentric Reference Frame	25
3.6.4 RK4 Orbit Prediction	26
3.6.5 Planetarium Floating Objects	27
4 Atmospheric Scattering	28
4.1 Mathematical Model of the Atmosphere	29
4.1.1 The Shader	29
4.1.2 The Vertex Shader	30
4.1.3 Rasterization	30

4.1.4	The Fragment Shader	31
4.2	Application to Atmospheric Scattering	31
4.2.1	Scattering Functions	32
4.3	The Mathematics of Atmospheric Scattering	33
5	Math and Physics: Ship Module	36
5.1	Spacecraft Axes	37
5.2	Reaction Control	37
5.3	Inertia Tensor: Rotation	38
5.4	Composite Centre of Mass	40
5.5	Composite Inertia Tensor	41
5.6	Torque	43
5.7	Translational Movement	44
5.8	Handling Position and Rotation of the Game Object	45
6	Crafting Dynamic Spaceships and Paths	46
6.1	Modular Ship Construction	46
6.1.1	The Goal	46
6.1.2	Defining Modules	46
6.1.3	Attachment Points	48
6.1.4	Control	49
6.2	Pathfinding	49
6.2.1	It's Raycasts All the Way Down	50
6.2.2	Object Avoidance	50
6.2.3	Movement	51
7	Future Work and Roadmap	52

1 Introduction

The overarching goal of *Endangered Orbits* is to engage deeply with the mathematics and physics of game design, specifically the rules that govern how objects in a scene move and rotate, built on the framework of a space-simulation game. The project is an attempt to demonstrate and bridge the gap between a computer-science and an information-technology perspective on game development: the “math” part.

Consequently, the foremost emphasis of *Endangered Orbits* is realistic physics. By adopting the real scales found in the cosmos, the simulation achieves a level of depth that, in several respects, is deeper than comparable space games in both spacecraft-movement physics and orbital mechanics. The second emphasis is graphical fidelity, realised through mathematical compute shaders that run natively on the GPU as a post-processing technique conceptually similar to ray tracing.

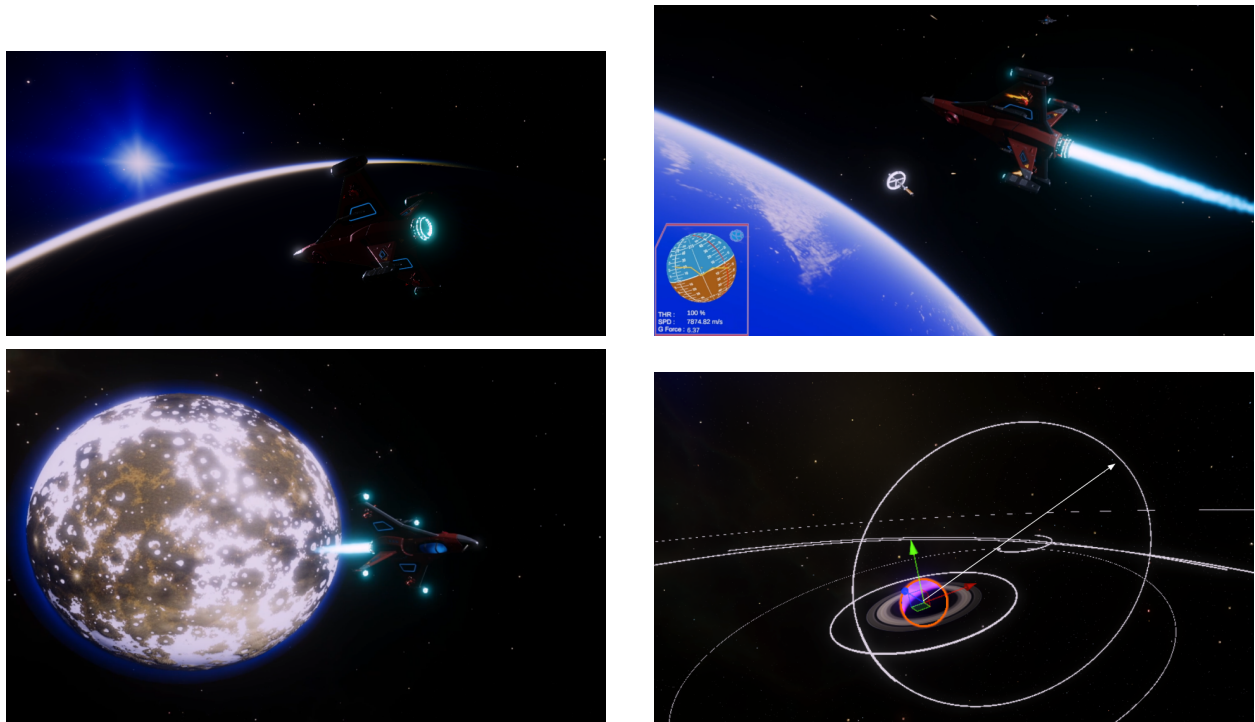


Figure 1. In-engine views of *Endangered Orbits*: predicted orbit lines and celestial bodies rendered alongside a real-scale spacecraft.

The game is organised into two interfacing subsystems that are rendered in separate layers to simulate scenes at vastly different scales:

- the **planetarium module**, which contains all orbital mechanics and all celestial bodies; and
- the **ship module**, which contains all ship-related objects and physics.

Both modules are designed to interface with each other, but are rendered in different layers using Unity’s layering system. Compositing the two layers produces the final game. The remainder of this report walks through the implementation of each module, the graphics pipeline behind the

atmospheres, and the gameplay systems for modular construction and pathfinding. Sections that contain substantial derivations are flagged in the text; the mathematics is presented in full notation throughout.

2 Overview of the Implementation

Several constraints arise from the decision to build a real-scale simulation in Unity. Unity is a cross-platform engine optimised for small- to mid-scale first-person, combat, or mobile games; its infrastructure is designed for those use cases. Making it behave as described above required significant work “under the hood”. The principal areas of challenge are scale and precision.

2.1 Scale of the Planetary System

There are two distinct kinds of scale. The first is pure *distance* between objects: the average separation between planets is measured in millions of kilometres. The second is the relative *size* of objects: a spaceship can be millions of times smaller than a planetary body. Neither extreme is supported by Unity’s engine out of the box.

The underlying problem is precision. To represent the locations of planets and a spaceship across distances spanning billions of kilometres, Unity’s transform system stores each object’s position as a single-precision floating-point number. At those magnitudes there is simply not enough precision to store the position of a ship. Worse, precision in game space is not uniformly distributed: objects near the game origin receive far more precision than objects far from it. From testing, beyond roughly 10,000 units (metres) from the origin precision becomes insufficient. This most severely affects the rendering pipeline, where vertex positions derived from a ship’s position suffer floating-point rounding errors that cause vertices to jitter rapidly.

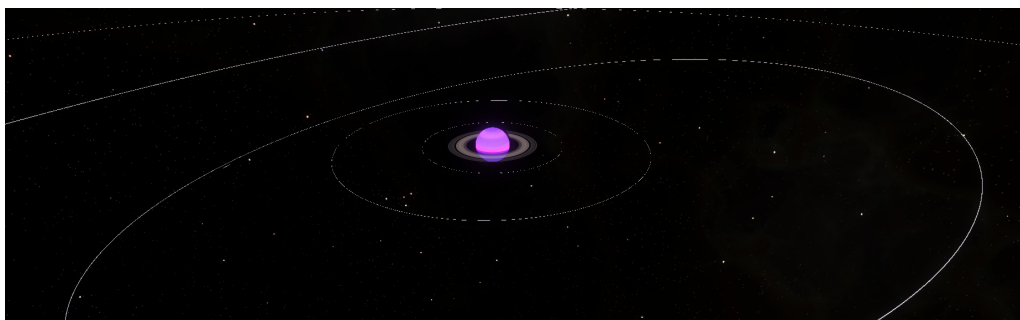


Figure 2. A wide view of the planetary system. Inter-planetary distances span millions of kilometres, far beyond the precision Unity’s single-precision transforms can represent natively.

2.2 Scale of Objects

Object scaling is a second major issue. To place a spaceship in the same scene as the celestial bodies, the ship would have to be scaled millions of times smaller. This again compounds the precision problem in the rendering pipeline, and it also distorts other rigid-body properties such as inertia, whose calculation assumes that one unit in game equals one metre in reality.

A three-pronged approach is used to solve precision and scale simultaneously:

1. **Double-precision mathematics.** All physics — positions, velocities, orbital integration — is computed in double precision *outside* the engine’s transform system, in true physical (world) coordinates.
2. **Floating origin.** A set of transformations remaps the default Cartesian grid to be *player-centric*, so the player is always at the game origin. This eliminates the unequal-precision problem, because the objects of interest are always near $(0, 0, 0)$.
3. **Separate scenes.** It is impossible to hold all objects in one scene at true scale, so each subsystem is rendered in its own scene/layer and the results are composited into the final image. This is also where the sense of scale and perspective is achieved.

With these transformations, game objects can be mapped from their world position (actual physical position) into editor position, where they can be scaled and rendered safely.

2.3 Realistic Behaviour of the Spaceship

Endangered Orbits lets the player attach and detach ship parts, each with its own shape, weight, and set of reaction-control (RCS) thrusters. To support realistic behaviour — for example asymmetric thrust — under arbitrary player-assembled configurations, Unity’s built-in rigid-body system is insufficient. Unity’s rigid-body component is built for a single object; multiple objects parented together do not have a combined inertia. Instead they are treated as one object with the *summed mass* but shaped like the parent collider only, which produces unrealistic behaviour.



Figure 3. A modular spacecraft firing thrusters near a gas giant. Each attached module contributes its own mass, shape, and reaction-control thrusters to the composite body.

To achieve realistic ship behaviour regardless of what the player attaches, Unity’s rigid-body system is recreated externally. Default rigid-body features are retained only to read per-module properties; these are then combined into a *composite inertia tensor*. The composite tensor lets us represent mathematically how an assembled body reacts to external forces such as a thrusting engine or the reaction-control system. The full derivation appears in Section 5.

2.4 Control of the Spaceship

Controlling a spacecraft in space is a non-trivial task. Unlike an aircraft, the player must control the craft across all six axes: three translational (x, y, z) and three rotational (x, y, z). Translation governs how the craft moves in 3-space (forward, up, left, and so on), while the rotational axes govern its pitch, roll, and yaw, much like an aircraft. Unity provides no built-in control for this; the motion is produced by firing tens or even hundreds of thrusters distributed around the ship. It is impossible for a player to micro-manage how each thruster fires.

We created a hybrid control scheme drawing inspiration from various space games and from real spacecraft. At its core, the player controls pitch and yaw with the mouse: the player simply points where they want the ship to face, and a proportional–integral–derivative (PID) system fires the thrusters required to reach that orientation. Translational movement is handled with the keyboard, similar to character movement in other games. In total the spacecraft uses **six closed-loop PID controllers**, one for each of its six axes of motion.

The following subsections summarise four gameplay systems that build on this control scheme; each is treated in full later in the report.

2.4.1 Modular Ship Construction

In *Endangered Orbits*, players build spaceships in real time, much like playing with building blocks. The idea was inspired by *Captain Forever*, taken a step further into full 3D. Each part — which we call a *module* — is designed to connect easily with others, forming different shapes and functions; modules can attach to or detach from one another during gameplay. To build a ship the player clicks to select a part, drags it over one of the attachment orbs that appear, and releases.



Figure 4. Modular construction in flight. Attachment points appear as orbs; dragging a module over an orb snaps it into place.

Taking this idea into 3D created two notable problems: deciding where to “float” a module while the player is holding it, and designing a control system that lets the player fly the ship and build it at the same time. These are addressed in Section 6.

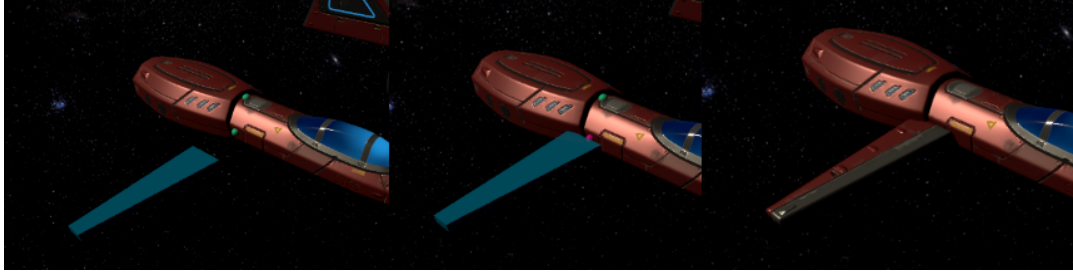


Figure 5. A module being attached to the hull during flight.

2.4.2 Pathfinding

Because there is no ground — nor any inherent “up” — in space, Unity’s built-in navigation cannot be used. This is not a severe limitation, however, because space is mostly empty. All NPCs fly directly toward their destination and use a raycast to test whether the direct route is blocked. If it is blocked, an intermediate waypoint that routes around the obstacle is found by inspecting the obstacle’s bounds.



Figure 6. A formation of NPC craft. Each NPC plans its own path and avoids collisions independently.

2.4.3 Object Avoidance

Compared with the pathfinding, the NPCs have a rather robust object-avoidance system, broken into three steps that run every update:

1. An NPC checks a short radius around itself with an overlap sphere to see whether anything is nearby, and computes a safer direction if necessary.
2. The NPC polls a layer containing all ships every update to see whether a collision with any of them is possible; if so, it tries to avoid it.
3. The NPC fires a sphere-cast in its direction of travel every update to check for obstacles it might fly into, so it can try to avoid them.

2.4.4 Applying Movement

To combine the instructions from these various movement steps, each step does not move the ship itself; instead it *returns* the way the ship should move for that task. In the physics step, the results from every step are summed, normalised, and sent to the PID controller to be translated into actual movement. This guarantees the steps do not compete with one another, and that the most important move gets priority — steps produce larger acceleration requests the more urgent their task is, so when the normalisation step runs, only the most important move survives.

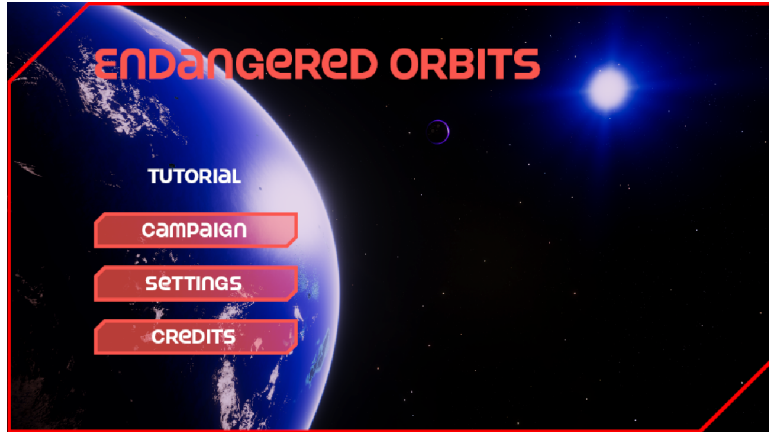


Figure 7. The finished start menu, with Campaign, Settings, and Credits. The user interface — start menu, credits, pause, and loading screens — is driven by dedicated scripts.

3 Math and Physics: Planetarium Module

The planetarium module is responsible for all code and objects relating to orbital mechanics, as well as the graphical elements — planets, atmospheres, and stars. Table 1 summarises its components.

Component	Key function	Source (excl. deps.)
Keplerian	Reference frames; convert Keplerian elements → state vectors and back; track time; floating origin.	Planetarium.cs, PlanetariumSingleton.cs, DrawOrbitLines.cs, Orbit.cs
Orbital integrator	Classical two-body solver; N -body orbit integrator.	Orbit.cs
Atmosphere	Real-time atmospheric scattering for planets.	Atmosphere.cs, Atmosphere.shader, Atmosphere.hlsl

Table 1. Components of the planetarium module.

3.1 Keplerian Component: Time

The core function is, first, to keep track of time. An initial time is set, and time is integrated in a fixed update. This takes into account time warp. Time is then used to compute the positions of

celestial bodies, whose orbits are determined with respect to time.

Listing 1. Initialising and integrating simulation time.

```

1 public DateTime initialDateTime = new DateTime(2023, 11, 30, 0, 0, 0, 0);
2
3 // ...
4 fixedTimeStep = Time.fixedDeltaTime * gameSpeed;
5 // Integrate game time.
6 currentDateTime = currentDateTime.AddSeconds(fixedTimeStep);

```

3.2 Keplerian Component: Reference Frame

Reference frames are important for all Keplerian orbits. Consider a heliocentric inertial frame in which a blue giant orbits the central star. The white grid lines lie parallel to the horizontal axis of game space; this is the parent body's **ecliptic plane** — the plane in which the orbit resides. Other planes can be defined: the vernal/equinox direction, and the body's **equatorial plane** (where, for example, its rings lie), which intersects the ecliptic plane. The definitions of the ecliptic planes live in `Orbit.cs` and are updated each frame from the local forward transform of the parent body.

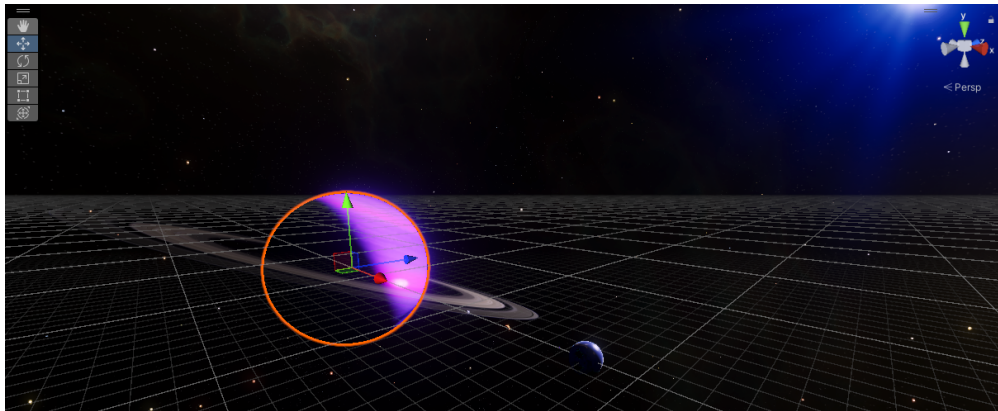


Figure 8. The heliocentric inertial frame in the editor. The red arrow marks the equatorial plane (and ring plane); the blue arrow points toward the parent or solstices.

Listing 2. Defining and updating the ecliptic basis.

```

1 // Orbital Planes
2 public Vector3d EclipticNormal = new Vector3d(0, 0, 1); // Normal of ecliptic
3 public Vector3d EclipticUp     = new Vector3d(0, 1, 0); // Up on ecliptic plane
4 public Vector3d EclipticRight  = new Vector3d(1, 0, 0); // Right on ecliptic plane
5
6 public void FixedUpdate()
7 {
8     if (!isBodySun)
9     {
10         EclipticNormal = Vector3d.toVector3d(parentBody.transform.forward);
11         EclipticUp     = Vector3d.toVector3d(parentBody.transform.up);
12         EclipticRight  = Vector3d.toVector3d(parentBody.transform.right);
13         thisCelestialAxis = Quaternion.LookRotation(transform.forward, transform.up);
14     }
15 }

```

```

15     updateInspectorDisplay();
16 }

```

State-vector reference frames. For each celestial or satellite object there are two reference frames: the *parent* frame and the *heliocentric* frame. The parent frame stores position and velocity relative to the object’s orbiting body (the Moon’s parent is the Earth, for example). The heliocentric frame stores position and velocity relative to the geometric centre of the Sun (or whichever body is heliocentric, which may itself be the blue giant). In `Orbit.cs`, `position` and `velocity` refer to the parent-centric frame, and `helio` denotes the heliocentric frame.

3.3 Keplerian Component: Orbital Elements to State Vector

`Orbit.cs` defines all Keplerian elements; these parameters are computed in real time (Figure 9).

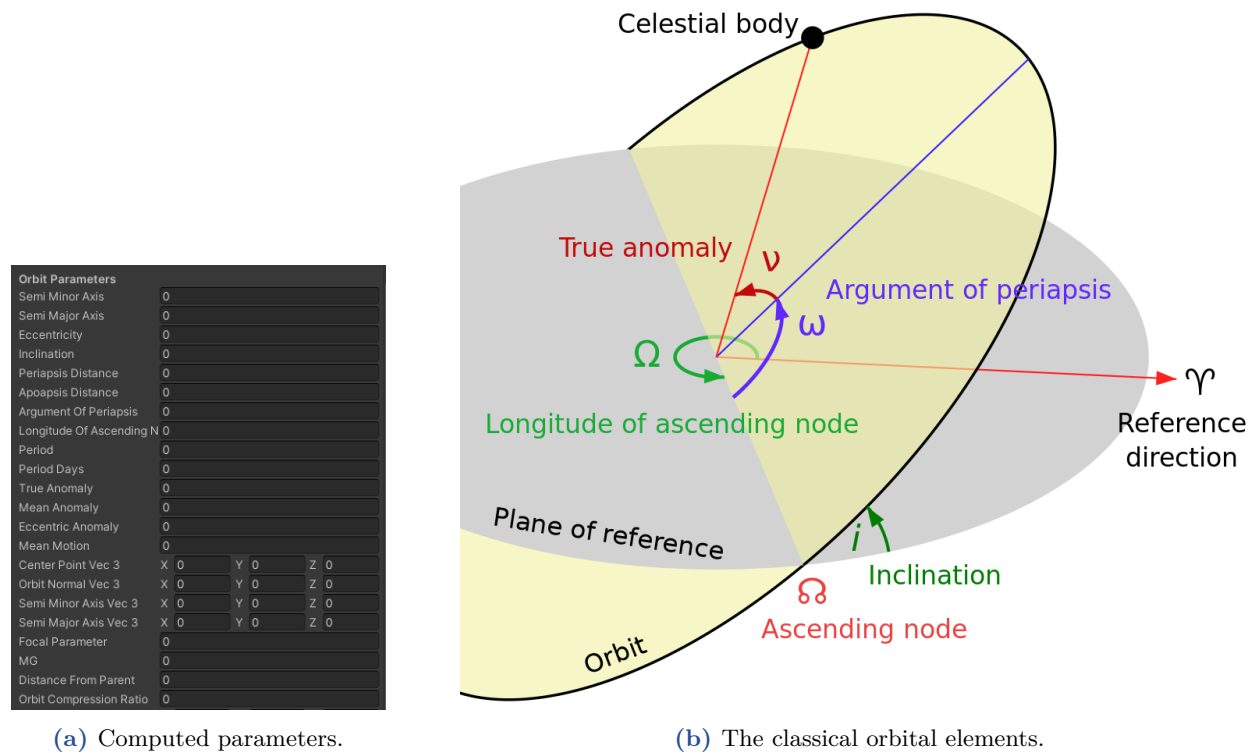


Figure 9. Orbital elements as exposed in the inspector (left) and illustrated in the standard reference diagram (right).

The elements are defined as follows.

Eccentricity (e)

the shape of the orbit ellipse. $e = 0$ is a perfectly circular orbit, while $e \geq 1$ is a hyperbolic or escape orbit. For reference, the Moon’s orbit around Earth is quite circular, with $e \approx 0.055$.

Semi-major axis (a)

the maximum “height” of the orbiting body relative to the geometric centre of the parent body.

(In reality the parent body wobbles, so this is more of an averaged centre.)

Semi-minor axis (b)

the smallest “height” of the orbiting body relative to the geometric centre of the parent body.

Apoapsis

a minus the planetary radius; often used to measure altitude above ground level (AGL).

Periapsis

b minus the planetary radius, or the closest distance the object reaches, relative to the parent’s AGL.

Semi-major / semi-minor axis vectors

a and b in vector form.

Inclination (i)

the angle of the orbital plane relative to the parent’s equatorial plane. $i = 0$ means the object orbits perfectly around the equator; $i = 90^\circ$ means the orbit passes over the poles.

Longitude of ascending node (LAN, Ω)

besides the tilt described by inclination, the orbit can also yaw around the parent’s polar axis; this yaw angle is the LAN. The reference direction is the parent’s ecliptic-right (red arrow).

Ascending node

the vector along which the orbit intersects the parent’s equatorial plane (going up).

Argument of periapsis (AOP, ω)

for an eccentric orbit, the highest and lowest points lie in a single direction. The direction at which the periapsis is located, measured relative to the ascending node, is the AOP.

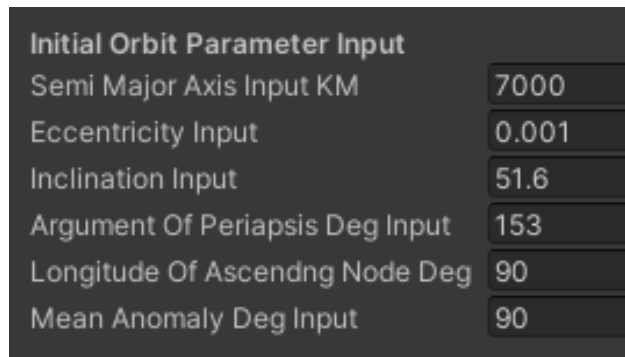
True anomaly (ν)

the angle of the body along the elliptical orbit, measured from the reference direction.

Mean anomaly (M)

like the true anomaly, but with the elliptical orbit “squished” into a circular one.

For each celestial or satellite object we need an initial starting condition. Rather than specify state vectors directly, we place the object in the scene by defining its orbit parameters (Figure 10); this guarantees the initialised state vectors are accurate. The following steps derive the state vectors from that input.



Initial Orbit Parameter Input	
Semi Major Axis Input KM	7000
Eccentricity Input	0.001
Inclination Input	51.6
Argument Of Periapsis Deg Input	153
Longitude Of Ascending Node Deg	90
Mean Anomaly Deg Input	90

Figure 10. Initial orbit-parameter input in the inspector.

Step 1 — standard gravitational parameter. Determine the standard gravitational parameter μ of the parent body:

$$\mu = G(M + m) \approx GM, \quad (1)$$

where G is the gravitational constant ($G = 6.67430 \times 10^{-11}$) and M is the mass of the parent body in kilograms. The satellite mass m is negligible compared with the parent, so it is dropped.

```
1 MG = parentBodyOrbit.mass * gravConst;
```

Step 2 — semi-minor from semi-major. The semi-minor axis follows from the eccentricity:

$$b = \begin{cases} a\sqrt{1 - e^2}, & e < 1 \quad (\text{ellipse}), \\ a\sqrt{e^2 - 1}, & e > 1 \quad (\text{hyperbola}), \end{cases} \quad (2)$$

where a is the semi-major axis input and e the eccentricity input.

```
1 // Handle Eccentricity
2 Eccentricity = EccentricityInput;
3 if (Eccentricity < 1.0)
4     SemiMinorAxis = SemiMajorAxis * Math.Sqrt(1 - Eccentricity * Eccentricity);
5 else if (Eccentricity > 1.0)
6     SemiMinorAxis = SemiMajorAxis * Math.Sqrt(Eccentricity * Eccentricity - 1);
7 else
8     SemiMajorAxis = 0;
```

Step 3 — ascending node and orbit-normal directions. We start from the parent's local red arrow (ecliptic-right). Given the LAN input and the parent's normal vector, the red arrow is rotated by the LAN angle about the normal to obtain the ascending node. The rotation uses Rodrigues' rotation formula: for a unit axis $\hat{\mathbf{u}} = (u_x, u_y, u_z)$ and angle θ , the rotation matrix is

$$\mathbf{R} = \begin{bmatrix} \cos \theta + u_x^2(1 - \cos \theta) & u_x u_y(1 - \cos \theta) - u_z \sin \theta & u_x u_z(1 - \cos \theta) + u_y \sin \theta \\ u_y u_x(1 - \cos \theta) + u_z \sin \theta & \cos \theta + u_y^2(1 - \cos \theta) & u_y u_z(1 - \cos \theta) - u_x \sin \theta \\ u_z u_x(1 - \cos \theta) - u_y \sin \theta & u_z u_y(1 - \cos \theta) + u_x \sin \theta & \cos \theta + u_z^2(1 - \cos \theta) \end{bmatrix}. \quad (3)$$

The rotated vector is then $\mathbf{R} \mathbf{v}$. The same construction, applied with the inclination input, yields the orbit normal.

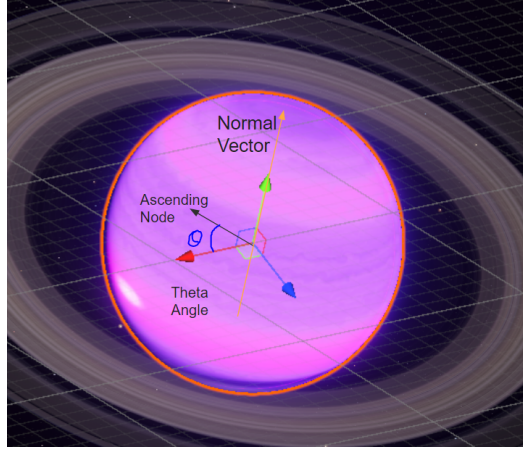


Figure 11. Normal vector, ascending node, and the rotation (θ) angle, shown in the editor.

Listing 3. Rodrigues rotation of a vector about an axis.

```

1 public static Vector3d RotateVectorByAngle(Vector3d v, double angleRad, Vector3d n)
2 {
3     double cosT = Math.Cos(angleRad);
4     double sinT = Math.Sin(angleRad);
5     double oneMinusCos = 1f - cosT;
6     // Rotation matrix:
7     double a11 = oneMinusCos * n.x * n.x + cosT;
8     double a12 = oneMinusCos * n.x * n.y - n.z * sinT;
9     double a13 = oneMinusCos * n.x * n.z + n.y * sinT;
10    double a21 = oneMinusCos * n.x * n.y + n.z * sinT;
11    double a22 = oneMinusCos * n.y * n.y + cosT;
12    double a23 = oneMinusCos * n.y * n.z - n.x * sinT;
13    double a31 = oneMinusCos * n.x * n.z - n.y * sinT;
14    double a32 = oneMinusCos * n.y * n.z + n.x * sinT;
15    double a33 = oneMinusCos * n.z * n.z + cosT;
16
17    // Equal to dot product
18    return new Vector3d(
19        v.x * a11 + v.y * a12 + v.z * a13,
20        v.x * a21 + v.y * a22 + v.z * a23,
21        v.x * a31 + v.y * a32 + v.z * a33);
22 }

```

```

1 ascendingNode = OrbitUtils.RotateVectorByAngle(ascendingNode,
2     longitudeOfAscendingNodeDeg * OrbitUtils.Deg2Rad, normal).normalized;
3 normal = OrbitUtils.RotateVectorByAngle(normal,
4     InclinationInput * OrbitUtils.Deg2Rad, ascendingNode).normalized;
5
6 OrbitNormal = -Vector3d.Cross(SemiMajorAxisBasis, SemiMinorAxisBasis).normalized;
7 OrbitNormalDotEclipticNormal = Vector3d.Dot(OrbitNormal, EclipticNormal);

```

The dot product of the orbit normal with the ecliptic normal is used to determine the direction (prograde or retrograde) of the orbit.

Step 4 — orbital period.

$$T = 2\pi\sqrt{\frac{a^3}{GM}}, \quad (4)$$

where a is the semi-major axis and T the period in seconds.

Step 5 — semi-major / semi-minor basis vectors. The ascending-node vector is rotated by the AOP into θ_2 ; the semi-major basis is then simply the resulting direction (equivalently, the cross product of periapsis and the parent normal axis).

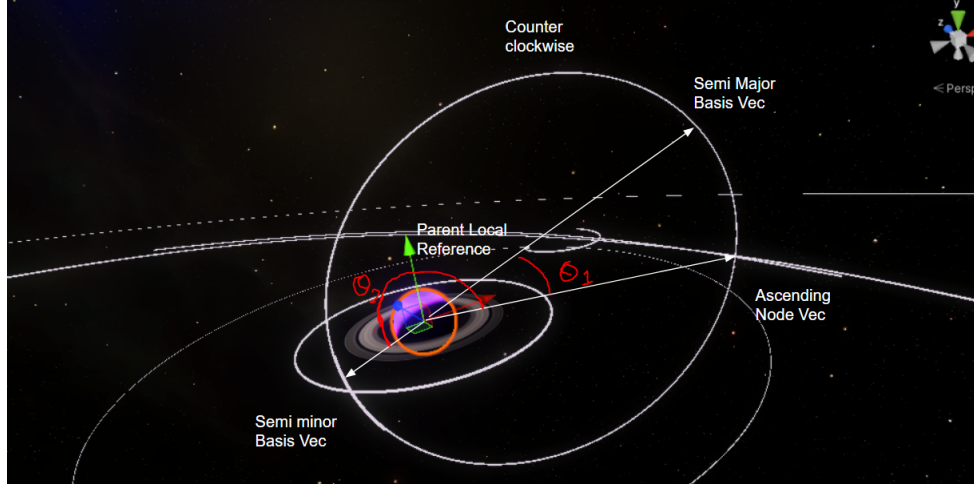


Figure 12. The semi-major and semi-minor basis vectors and the ascending-node vector, visualised in the editor.

```

1 Periapsis = ascendingNode;
2 Periapsis = OrbitUtils.RotateVectorByAngle(Periapsis,
3     argumentOfPeriapsisDegInput * OrbitUtils.Deg2Rad, normal).normalized;
4
5 SemiMajorAxisBasis = Periapsis;
6 SemiMinorAxisBasis = Vector3d.Cross(Periapsis, normal);

```

The basis vectors are direction vectors only. The vector forms of the semi-major and semi-minor axes, and the apoapsis/periapsis distances, are

$$\mathbf{a} = \hat{\mathbf{a}}_{\text{basis}} a, \quad \mathbf{b} = \hat{\mathbf{b}}_{\text{basis}} b, \quad (5)$$

$$r_{\text{apo}} = \|\mathbf{a}\| - R_{\text{parent}}, \quad r_{\text{peri}} = \|\mathbf{b}\| - R_{\text{parent}}, \quad (6)$$

where R_{parent} is the parent body radius. If $e \geq 1$, the semi-major axis and apoapsis do not exist.

Step 6 — eccentric anomaly from mean anomaly. For an elliptical orbit ($e < 1$) we use Kepler's equation,

$$M = E - e \sin E, \quad (7)$$

where M is the mean anomaly, e the eccentricity, and E the eccentric anomaly. Since the user inputs a mean anomaly and the equation is transcendental in E , there is no closed form, so we

solve it with the Newton–Raphson method. Given a tolerance ε , we iterate

$$E_{n+1} = E_n - \frac{f(E_n)}{f'(E_n)} = E_n - \frac{E_n - |e| \sin(E_n) - M}{1 - |e| \cos(E_n)}, \quad (8)$$

where E_n is the eccentric anomaly at iteration n . In practice the iteration count is chosen from the eccentricity: a reasonable count ranges from 2 for near-circular orbits to 6 for near-hyperbolic ones.¹

Listing 4. Newton–Raphson Kepler solver (elliptic case).

```

1 public static double KeplerSolver(double meanAnomaly, double eccentricity)
2 {
3     // Iterations range from 2 to 6 when eccentricity is in [0, 1].
4     int iterations = (int)(Math.Ceiling((eccentricity + 0.7d) * 1.25d)) << 1;
5
6     double m = meanAnomaly;
7     double esinE, ecosE, deltaE, n;
8     for (int i = 0; i < iterations; i++)
9     {
10         esinE = eccentricity * Math.Sin(m);
11         ecosE = eccentricity * Math.Cos(m);
12         deltaE = m - esinE - meanAnomaly;
13         n = 1.0 - ecosE;
14         m += -5d * deltaE /
15             (n + Math.Sign(n) * Math.Sqrt(Math.Abs(16d * n * n - 20d * deltaE * esinE)));
16     }
17     return m;
18 }

```

For a hyperbolic orbit ($e > 1$) we use the hyperbolic anomaly F , defined through the hyperbolic Kepler equation:

$$M_h = e \sinh F - F, \quad (9)$$

$$f(F) = e \sinh F - F - M_h = 0, \quad (10)$$

$$f'(F) = e \cosh F - 1, \quad (11)$$

where M_h is the mean anomaly and a tolerance of $\varepsilon \approx 10^{-8}$ is typical.

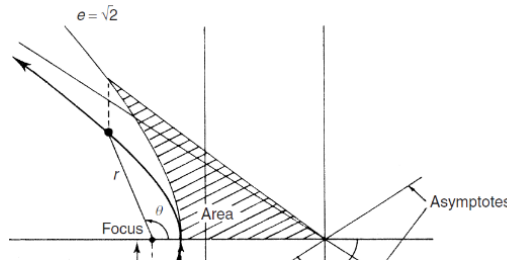


Figure 13. The hyperbolic case ($e = \sqrt{2}$): the hyperbolic anomaly is the swept area F , bounded by the asymptotes.

¹Iteration scheme adapted from <https://scicomp.stackexchange.com/questions/40700/>.

Listing 5. Kepler solver, hyperbolic case (Danby initial guess).

```

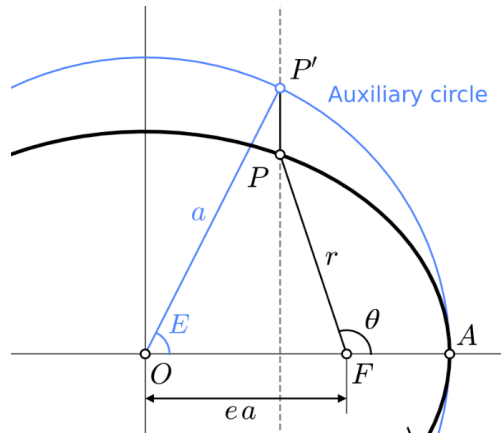
1 public static double KeplerSolverHyperbolicCase(double meanAnomaly, double eccentricity)
2 {
3     double delta = 1d;
4     // Danby guess.
5     double F = Math.Log(2d * Math.Abs(meanAnomaly) / eccentricity + 1.8d);
6     if (double.IsNaN(F) || double.IsInfinity(F))
7         return meanAnomaly;
8
9     while (delta > 1e-8 || delta < -1e-8)
10    {
11        delta = (eccentricity * Math.Sinh(F) - F - meanAnomaly) /
12              (eccentricity * Math.Cosh(F) - 1d);
13        F -= delta;
14    }
15    return F;
16 }

```

Step 7 — position from eccentric anomaly. Applying the Pythagorean theorem on the auxiliary circle (Figure 14), with O the centre, F the focus (parent origin), a the semi-major axis, $|OA|$ and $|ea|$ the marked lengths, we have

$$|OA| = a, \quad \cos E = \frac{|ea|}{a}, \quad r = a(1 - e \cos E). \quad (12)$$

To obtain the position relative to the parent, take $F = 0$; the position vector P' is then the semi-major axis basis (pointing from F to P) scaled by r .

**Figure 14.** Geometry relating the eccentric anomaly E , the auxiliary circle, and the radius r .**Listing 6.** Position at a given eccentric anomaly.

```

1 public Vector3d GetCentralPositionAtEccentricAnomaly(double eccentricAnomaly)
2 {
3     if (Eccentricity < 1.0)
4     {
5         Vector3d result = new Vector3d(Math.Sin(eccentricAnomaly) * SemiMinorAxis,
6                                         -Math.Cos(eccentricAnomaly) * SemiMajorAxis);
7     }
8 }

```

```

7     return -SemiMinorAxisBasis * result.x - SemiMajorAxisBasis * result.y;
8 }
9 else if (Eccentricity > 1.0)
10 {
11     Vector3d result = new Vector3d(Math.Sinh(eccentricAnomaly) * SemiMinorAxis,
12                                   Math.Cosh(eccentricAnomaly) * SemiMajorAxis);
13     return -SemiMinorAxisBasis * result.x - SemiMajorAxisBasis * result.y;
14 }
15 else
16 {
17     var pos = new Vector3d(
18         PeriapsisDistance * Math.Sin(eccentricAnomaly) / (1.0 + Math.Cos(eccentricAnomaly))
19         ,
20         PeriapsisDistance * Math.Cos(eccentricAnomaly) / (1.0 + Math.Cos(eccentricAnomaly))
21     );
22     return -SemiMinorAxisBasis * pos.x + SemiMajorAxisBasis * pos.y;
23 }

```

For an ellipse we use the ordinary trigonometric functions; for a hyperbola we use the hyperbolic ones.

Step 8 — velocity from true anomaly. The velocity calculation uses the true anomaly, so we first convert the eccentric anomaly E to the true anomaly ν :

$$\cos \nu = \frac{\cos E - e}{1 - e \cos E} \quad (\text{elliptic case; use hyperbolic functions otherwise}). \quad (13)$$

Listing 7. Eccentric-to-true anomaly conversion.

```

1 public static double ConvertEccentricToTrueAnomaly(double eccentricAnomaly, double eccentricity
2 )
3 {
4     if (eccentricity < 1.0)
5     {
6         double cosE = Math.Cos(eccentricAnomaly);
7         double tAnom = Math.Acos((cosE - eccentricity) / (1d - eccentricity * cosE));
8         if (eccentricAnomaly > PI) tAnom = PI_2 - tAnom;
9         return tAnom;
10    }
11    else if (eccentricity > 1.0)
12    {
13        double tAnom = Math.Atan2(
14            Math.Sqrt(eccentricity * eccentricity - 1d) * Math.Sinh(eccentricAnomaly),
15            eccentricity - Math.Cosh(eccentricAnomaly));
16        return tAnom;
17    }
18    else return eccentricAnomaly;
19 }

```


The velocity is then obtained in the perifocal (PQW) reference frame:

$$\mathbf{v}_{\text{perifocal}} = \begin{bmatrix} -\sqrt{\frac{\mu}{p}} \sin f \\ \sqrt{\frac{\mu}{p}} (e + \cos f) \\ 0 \end{bmatrix}, \quad (14)$$

where f is the true anomaly, p the semi-latus rectum, μ the parent's standard gravitational parameter, and e the eccentricity.

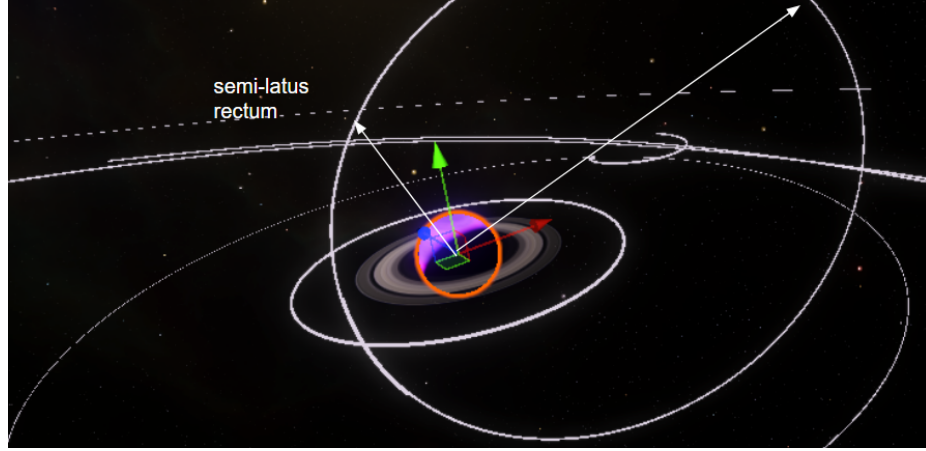


Figure 15. The semi-latus rectum in the perifocal frame, visualised in-engine.

3.4 State Vector to Orbital Elements

The same operation must also be performed in reverse: an object's state vector (position and velocity) is converted to orbital parameters. This is used primarily by the N -body simulation, where after each integration step the resulting position and velocity are converted back to orbital parameters for display or trajectory planning.

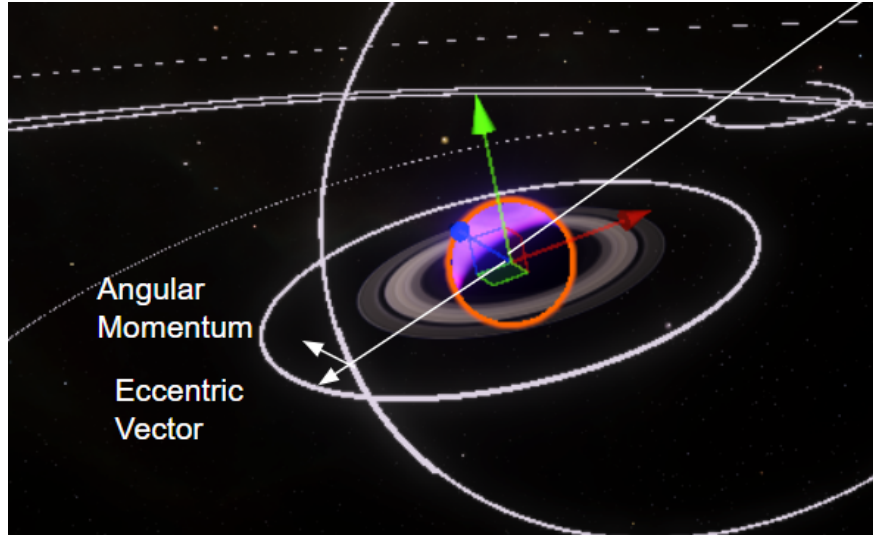


Figure 16. An N -body trajectory whose state vectors are converted back to Keplerian elements each step.

Step 1 — the eccentricity vector. The eccentricity vector points from the parent body’s centre to the periapsis of the orbit; its magnitude is the eccentricity. It is a function of position and velocity via the specific angular momentum:

$$\mathbf{e} = \frac{\mathbf{v} \times \mathbf{h}}{\mu} - \frac{\mathbf{r}}{\|\mathbf{r}\|}, \quad (15)$$

where $\mathbf{h} = \mathbf{r} \times \mathbf{v}$ is the specific angular momentum, $\mathbf{r}$ and $\mathbf{v}$ are the parent-relative position and velocity, and $\mu = GM$. Note that position and velocity here are relative to the parent body only.

```
1 eccVector = Vector3d.Cross(velocity, angularMomentumVector) / MG
2   - position / distanceFromParent;
```

Step 2 — semi-major and semi-minor axes. The focal parameter (distance from the directrix to the focus) is

$$p = \frac{\|\mathbf{h}\|^2}{\mu}. \quad (16)$$

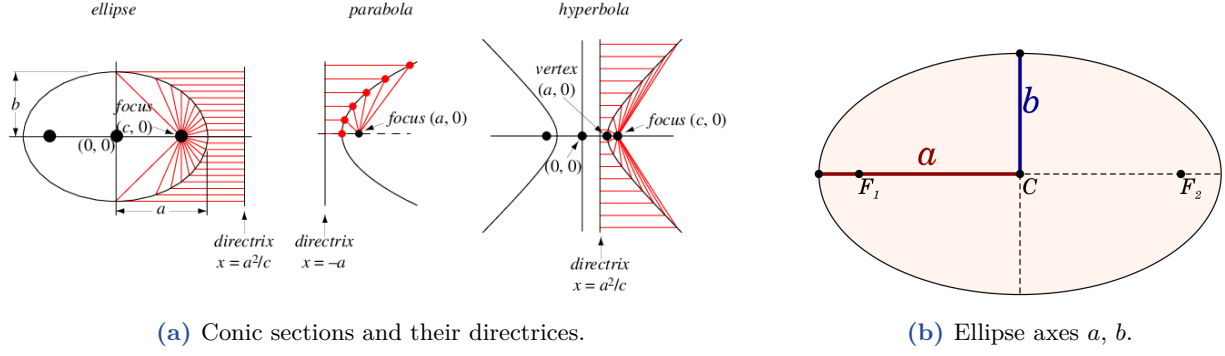


Figure 17. The conic that an orbit traces depends on its eccentricity; the directrix and focal parameter relate the geometry to the angular momentum.

```
1 FocalParameter = angularMomentumVector.sqrMagnitude / MG;
```

The semi-minor basis is the cross product of the angular-momentum vector with the eccentricity vector; the semi-major basis is then this rotated by 90° about the orbit normal:

```
1 SemiMinorAxisBasis = Vector3d.Cross(angularMomentumVector, -eccVector).normalized;
2 SemiMajorAxisBasis = Vector3d.Cross(OrbitNormal, SemiMinorAxisBasis).normalized;
```

The axis distances follow from the angular momentum and eccentricity:

$$\|\mathbf{h}\| = \sqrt{a\mu(1 - e^2)}, \quad b = a\sqrt{1 - e^2}, \quad (17)$$

where a is the semi-major distance, b the semi-minor distance, e the eccentricity, and $\mu = GM$ the parent's gravitational parameter.

Apogee and perigee. Since the distance between apogee and perigee is $2a$, with the centre-point position vector $\mathbf{c} = \hat{\mathbf{a}}_{\text{basis}} ae$ (relative to the parent origin):

$$\mathbf{r}_{\text{apo}} = \mathbf{c} - \hat{\mathbf{a}}_{\text{basis}} a, \quad \mathbf{r}_{\text{peri}} = \mathbf{c} + \hat{\mathbf{a}}_{\text{basis}} a. \quad (18)$$

The apogee and perigee distances are the magnitudes of these vectors minus the parent radius.

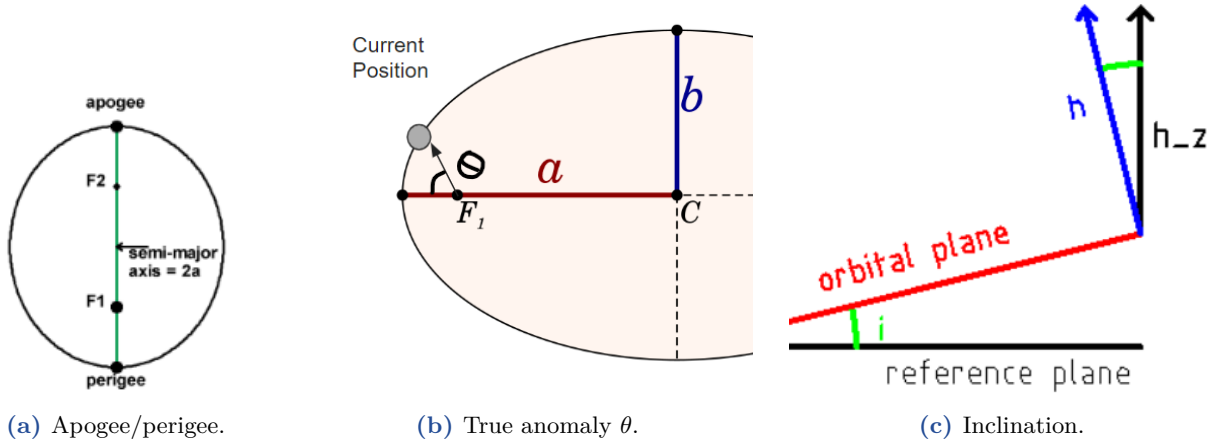


Figure 18. Geometry used to recover the true anomaly and inclination from a state vector.

Step 3 — true anomaly. The true anomaly is the angle between the current position vector (from the focus $(0,0,0)$ in the inertial frame) and the semi-major axis basis direction:

```
1 TrueAnomaly = Vector3d.Angle(position, SemiMajorAxisBasis) * OrbitUtils.Deg2Rad;
```

Step 4 — inclination. With the orbit normal $\hat{\mathbf{h}}$ and the parent's equatorial up axis $\hat{\mathbf{h}}_z$, the inclination is

$$i = \arccos(\hat{\mathbf{h}} \cdot \hat{\mathbf{h}}_z). \quad (19)$$

```
1 // Inclination
2 Inclination = Mathd.Acos(Vector3d.Dot(OrbitNormal, EclipticUp)) * Mathf.Rad2Deg;
```

Step 5 — longitude of ascending node. The ascending-node vector $\mathbf{n}$ is the intersection of the orbital plane and the parent's equatorial plane; its sense depends on whether the orbit is prograde or retrograde. With the ascending-node direction and the parent's reference direction (ecliptic-right) known, the angle between them is

$$\Omega' = \arccos(\hat{\mathbf{n}} \cdot \hat{\mathbf{e}}_{\text{right}}). \quad (20)$$

```
if (Vector3d.Dot(Vector3d.Cross(ascNodeDir, EclipticRight), EclipticNormal) >= 0)
{
    LongitudeOfAscendingNode = OrbitUtils.PI_2 - LANangle;
} else
{
    LongitudeOfAscendingNode = LANangle;
}
```

Figure 19. Recovering the LAN: the dot product gives the magnitude of the angle; a cross product disambiguates which side it lies on.

Because $\arccos$ returns an angle $< 180^\circ$, the quadrant must be resolved for LANs above 180° . Taking the cross product of the ascending node and the reference direction yields a third vector perpendicular to both; its dot product with the equatorial up direction is positive or negative depending on the side, which selects the branch:

```

1 var LANangle = Mathd.Acos(Vector3d.Dot(ascNodeDir, EclipticRight));
2
3 if (Vector3d.Dot(Vector3d.Cross(ascNodeDir, EclipticRight), EclipticNormal) >= 0)
4     LongitudeOfAscendingNode = OrbitUtils.PI_2 - LANangle;
5 else
6     LongitudeOfAscendingNode = LANangle;

```

3.5 N-Body Integrator

In `Orbit.cs` the user can select either a simple Keplerian orbit or a more complex N -body integrator. For the player ship in particular, an N -body integrator is the preferred orbit propagator. To support multiple time scales — such as time warps while maintaining accuracy — we implemented Newton’s equations together with fourth- and eighth-order Runge–Kutta schemes.

The core operation is Newton’s law of universal gravitation: the force acting on one object is proportional to the parent’s gravitational parameter and inversely proportional to the squared distance. In scalar and vector form,

$$F = G \frac{m_1 m_2}{r^2}, \quad \mathbf{F}_{21} = -G \frac{m_1 m_2}{\|\mathbf{r}_{21}\|^2} \hat{\mathbf{r}}_{21}. \quad (21)$$

To account for all bodies, the vector form is summed over N celestial bodies. Because the spacecraft mass is insignificant compared with a planet, m_2 is taken to be zero for the craft.

Listing 8. Summed N -body acceleration over one discrete step.

```

1 public Vector3d getVelocityNbodyFuture(double _dt, Vector3d position)
2 {
3     Vector3d _acceleration = Vector3d.zero;
4     foreach (Orbit otherCelestialBody in _planetarium.celestialObjectOrbit)
5     {
6         if (_planetarium.celestialBody.Count > 1 && otherCelestialBody != this)
7         {
8             // Acceleration contribution from each body.
9             double sqrDst = (otherCelestialBody.futureHelioCentricPosition - position).
            sqrMagnitude;
10            Vector3d forceDir = (otherCelestialBody.futureHelioCentricPosition - position).
            normalized;
11            _acceleration += (forceDir * otherCelestialBody.MG) / sqrDst;
12        }
13    }
14    return _acceleration * _dt;
15 }

```

A naive Euler integrator must use very small time steps (below 0.01 s) for acceptable accuracy, because integration error accumulates and orbits magically gain or lose energy. Runge–Kutta methods let us use much larger steps for the same accuracy.

3.6 Numerical Runge–Kutta Methods

3.6.1 RK4 Scheme

For low time-warp settings, RK4 provides reasonable accuracy; the planetarium module currently uses RK4 for all integration steps. The scheme first computes four coefficients,

$$k_1 = f(t_n, y_n), \quad (22)$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + k_1 \frac{h}{2}\right), \quad (23)$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + k_2 \frac{h}{2}\right), \quad (24)$$

$$k_4 = f(t_n + h, y_n + k_3 h), \quad (25)$$

where h is the time step (the explicit t_n dependence is unused here). These are combined as

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \quad (26)$$

```
public Vector3d getVelocityFromBodyFuture(double _dt, Vector3d position)
{
    Vector3d _acceleration = Vector3d.zero;
    foreach (Orbit otherCelestialBody in _planetarium.celestialObjectOrbit)
    {
        if (_planetarium.celestialObjectOrbit.Count > 1 && otherCelestialBody != this)
        {
            // Calculate Acceleration for each body.
            double sqrDst = ((otherCelestialBody.futureHeliocentricPosition - (position)).sqrMagnitude);
            Vector3d forceDir = (otherCelestialBody.futureHeliocentricPosition - (position)).normalized;
            _acceleration += (forceDir * otherCelestialBody.MG) / sqrDst;
        }
    }
    return _acceleration * _dt;
}
```

Figure 20. RK4 evaluates four sub-step gradients $k_1, \dots, k_4$ within a single step and combines them, approximating the accuracy of a much finer Euler integration.

To integrate both position and velocity, each coefficient is split into a velocity component $k_{n,v}$ and a position component $k_{n,r}$ (writing $\mathbf{a}$ for acceleration and $\mathbf{r}$ for position):

$$\mathbf{k}_{1,v} = \mathbf{a}(\mathbf{r}_i), \quad \mathbf{k}_{1,r} = \mathbf{v}_i, \quad (27)$$

$$\mathbf{k}_{2,v} = \mathbf{a}\left(\mathbf{r}_i + \mathbf{k}_{1,v} \frac{h}{2}\right), \quad \mathbf{k}_{2,r} = \mathbf{v}_i + \mathbf{k}_{1,v} \frac{h}{2}, \quad (28)$$

$$\mathbf{k}_{3,v} = \mathbf{a}\left(\mathbf{r}_i + \mathbf{k}_{2,v} \frac{h}{2}\right), \quad \mathbf{k}_{3,r} = \mathbf{v}_i + \mathbf{k}_{2,v} \frac{h}{2}, \quad (29)$$

$$\mathbf{k}_{4,v} = \mathbf{a}(\mathbf{r}_i + \mathbf{k}_{3,v} h), \quad \mathbf{k}_{4,r} = \mathbf{v}_i + \mathbf{k}_{3,v} h. \quad (30)$$

The updated velocity and position are then

$$\mathbf{v}_{i+1} = \mathbf{v}_i + \frac{h}{6}(\mathbf{k}_{1,v} + 2\mathbf{k}_{2,v} + 2\mathbf{k}_{3,v} + \mathbf{k}_{4,v}), \quad (31)$$

$$\mathbf{r}_{i+1} = \mathbf{r}_i + \frac{h}{6}(\mathbf{k}_{1,r} + 2\mathbf{k}_{2,r} + 2\mathbf{k}_{3,r} + \mathbf{k}_{4,r}). \quad (32)$$

Listing 9. RK4 integrator for coupled position and velocity.

```

1 public (Vector3d, Vector3d) RungeKutta_4(Vector3d k_v, Vector3d k_x, double _dt)
2 {
3     // K1
4     Vector3d k_1v = k_v + getVelocityNbodyFuture(_dt, k_x);
5     Vector3d k_1x = k_x + (_dt * k_1v);
6     // K2
7     Vector3d k_2v = k_v + getVelocityNbodyFuture(_dt / 2, (_dt / 2) * k_1x);
8     Vector3d k_2x = k_x + ((_dt / 2) * k_1v);
9     // K3
10    Vector3d k_3v = k_v + getVelocityNbodyFuture(_dt / 2, (_dt / 2) * k_2x);
11    Vector3d k_3x = k_x + ((_dt / 2) * k_2v);
12    // K4
13    Vector3d k_4v = k_v + getVelocityNbodyFuture(_dt, _dt * k_3v);
14    Vector3d k_4x = k_x + (_dt * k_3v);
15    // Composite
16    Vector3d k_v_new = k_v + (_dt / 6) * (k_1v + 2 * k_2v + 2 * k_3v + k_4v);
17    Vector3d k_x_new = k_x + (_dt / 6) * (k_1x + 2 * k_2x + 2 * k_3x + k_4x);
18
19    return (k_x_new, k_v_new);
20 }

```

3.6.2 RK8 Scheme

For larger time steps — for example drawing orbit-line predictions for interplanetary travel — a higher-order scheme is needed. RK8 is similar in principle to RK4 but uses eight stages. The stage nodes a_i and weights $b_{i,j}$ are given in Table 2.

i	a_i	$\sum_{j=0}^{i-1} b_{i,j}$
1	4/27	4/27
2	2/9	$\frac{1}{18}(1 + 3)$
3	1/3	$\frac{1}{12}(1 + 0 + 3)$
4	1/2	$\frac{1}{8}(1 + 0 + 0 + 3)$
5	2/3	$\frac{1}{54}(13 + 0 - 27 + 42 + 8)$
6	1/6	$\frac{1}{4320}(389 + 0 - 54 + 966 - 824 + 243)$
7	1	$\frac{1}{20}(-234 + 0 + 81 - 1164 + 656 - 122 + 800)$
8	5/6	$\frac{1}{288}(-127 + 0 + 18 - 678 + 456 - 9 + 576 + 4)$
9	1	$\frac{1}{820}(1481 + 0 - 81 + 7104 - 3376 + 72 - 5040 - 60 + 720)$
$\sum_i c_i = \frac{1}{840}(41 + 0 + 0 + 27 + 272 + 27 + 216 + 0 + 216 + 41)$		

Table 2. Butcher tableau coefficients for the eighth-order Runge–Kutta scheme.**Listing 10.** RK8 integrator (six-dimensional state vector).

```

1 // Runge-Kutta 8th order integrator

```

```

2 public (Vector3d, Vector3d) RungeKutta_8(Vector3d _initialPosition, Vector3d _initialVelocity,
   double _dt)
3 {
4     Vector6d y = new Vector6d(_initialPosition, _initialVelocity);
5
6     Vector6d k_1 = getAccelerationNbodyFuture(0, y);
7     Vector6d k_2 = getAccelerationNbodyFuture(_dt * (4 / 27), y + ((_dt * 4 / 27) * k_1));
8     Vector6d k_3 = getAccelerationNbodyFuture(_dt * (2 / 9), y + (_dt / 18) * (k_1 + 3 *
   k_2));
9     Vector6d k_4 = getAccelerationNbodyFuture(_dt * (1 / 3), y + (_dt / 12) * (k_1 + 3 *
   k_3));
10    Vector6d k_5 = getAccelerationNbodyFuture(_dt * (1 / 2), y + (_dt / 8) * (k_1 + 3 *
   k_4));
11    Vector6d k_6 = getAccelerationNbodyFuture(_dt * (2 / 3), y + (_dt / 54) * (13 * k_1 -
   27 * k_3 + 42 * k_4 + 8 * k_5));
12    Vector6d k_7 = getAccelerationNbodyFuture(_dt * (1 / 6), y + (_dt / 4320) * (389 * k_1 -
   54 * k_3 + 966 * k_4 - 824 * k_5 + 243 * k_6));
13    Vector6d k_8 = getAccelerationNbodyFuture(_dt, y + (_dt / 20) * (-234 * k_1 +
   81 * k_3 - 1164 * k_4 + 656 * k_5 - 122 * k_6 + 800 * k_7));
14    Vector6d k_9 = getAccelerationNbodyFuture(_dt * (5 / 6), y + (_dt / 288) * (-127 * k_1 +
   18 * k_3 - 678 * k_4 + 456 * k_5 - 9 * k_6 + 576 * k_7 + 4 * k_8));
15    Vector6d k_10 = getAccelerationNbodyFuture(_dt, y + (_dt / 820) * (1481 * k_1 -
   81 * k_3 + 7104 * k_4 - 3376 * k_5 + 72 * k_6 - 5040 * k_7 - 60 * k_8 + 720 * k_9));
16
17    Vector6d y_result = y + _dt / 840 * (41 * k_1 + 27 * k_4 + 272 * k_5 + 27 * k_6 + 216 * k_7
   + 216 * k_9 + 41 * k_10);
18
19    return (y_result.getPosition(), y_result.getVelocity());
20 }

```

3.6.3 Heliocentric Reference Frame

For the integrator to account for all objects, the integration must be performed in a single frame that contains every celestial body — the heliocentric frame. During initialisation, the computed position and velocity of an object are expressed in its parent-centric frame (the equatorial frame of its parent). To convert to heliocentric, we walk the tree from the current object up through each parent, summing every parent-centric position and velocity along the way.

Listing 11. Recursively accumulating parent contributions into the heliocentric frame.

```

1 public void getHelioPosition()
2 {
3     parentOrbits = findAllParentBody(transform, "celestialBody");
4
5     // Update heliocentric position and velocity.
6     helioCentricPosition = position;
7     helioCentricVelocity = velocity;
8
9     foreach (Orbit thisParent in parentOrbits)
10    {
11        helioCentricPosition += thisParent.position;
12        helioCentricVelocity += thisParent.velocity;
13    }
14 }

```


3.6.4 RK4 Orbit Prediction

To predict a future orbit — and thus draw orbit lines — the object’s position must be integrated forward to a defined future time. The positions of all celestial bodies must also be advanced to the same future time. Because a celestial body follows a classical Keplerian orbit, its future position is obtained directly from its anomaly.

Since the period T is already known, the mean motion gives the angular displacement of the mean anomaly over an elapsed time:

```
1 futureMeanAnomaly = MeanAnomaly + (MeanMotion * elapsedTimeS);
```

The future mean anomaly is then converted to a future eccentric anomaly with the Newton–Raphson method described earlier.

Listing 12. Future eccentric anomaly for a given elapsed time.

```
1 public double getFutureEccentricAnomaly(double elapsedTimeS)
2 {
3     double futureEccentricAnomaly = 0;
4     futureMeanAnomaly = MeanAnomaly + (MeanMotion * elapsedTimeS);
5
6     if (Eccentricity < 1.0)
7     {
8         futureMeanAnomaly %= OrbitUtils.PI_2;
9         if (futureMeanAnomaly < 0)
10             futureMeanAnomaly = OrbitUtils.PI_2 - futureMeanAnomaly;
11         futureEccentricAnomaly = OrbitUtils.KeplerSolver(futureMeanAnomaly, Eccentricity);
12     }
13     else if (Eccentricity > 1.0)
14     {
15         futureEccentricAnomaly = OrbitUtils.KeplerSolverHyperbolicCase(futureMeanAnomaly,
16             Eccentricity);
17     }
18     else
19     {
20         futureEccentricAnomaly = OrbitUtils.ConvertMeanToEccentricAnomaly(futureMeanAnomaly,
21             Eccentricity);
22     }
23     return futureEccentricAnomaly;
24 }
```

The future eccentric anomaly yields the future position, which then seeds the orbit- prediction integrator (Figure 21).

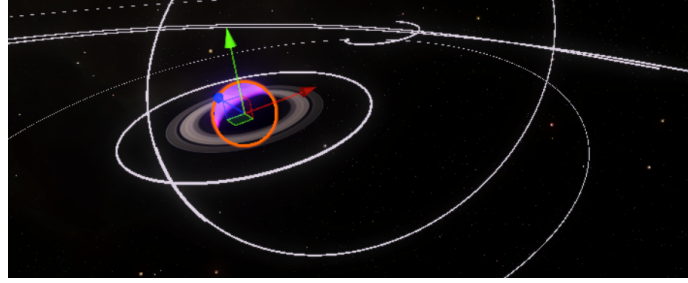


Figure 21. Predicted trajectories drawn by integrating each body forward in time.

Listing 13. Propagating both Keplerian and N -body orbits forward (in `DrawOrbitLines.cs`).

```

1 private void propagateNbodyOrbit(float timeFromCurrentS)
2 {
3     foreach (Orbit thisOrbit in _planetarium.celestialObjectOrbit)
4     {
5         if (!thisOrbit.isBodySun && thisOrbit._orbitType == Orbit.orbitType.Two_Body)
6         {
7             double futureEccVec = thisOrbit.getFutureEccentricAnomaly(timeFromCurrentS);
8             thisOrbit.futurePosition = thisOrbit.GetFocalPositionAtEccentricAnomaly(
9                 futureEccVec);
10            thisOrbit.futureHelioCentricPosition =
11                thisOrbit.futurePosition + thisOrbit.parentBodyOrbit.helioCentricPosition;
12        }
13    }
14    foreach (Orbit thisNbodyOrbit in _planetarium.satelliteObjectOrbit)
15    {
16        thisNbodyOrbit.IntegrateFuture(nBodySegmentLength);
17        thisNbodyOrbit.futurePosition =
18            thisNbodyOrbit.futureHelioCentricPosition - thisNbodyOrbit.parentBodyOrbit.
19            futureHelioCentricPosition;
20
21        if (thisNbodyOrbit._referenceFrame == Orbit.referenceFrame.Heliocentric)
22            thisNbodyOrbit.nBodyOrbitPoints.Add(thisNbodyOrbit.futureHelioCentricPosition);
23        if (thisNbodyOrbit._referenceFrame == Orbit.referenceFrame.Parentcentric)
24            thisNbodyOrbit.nBodyOrbitPoints.Add(thisNbodyOrbit.futurePosition);
25    }
26 }

```

3.6.5 Planetarium Floating Objects

In the planetarium, exactly one object — the player ship — is fixed at the floating origin. The player-ship anchor is a placeholder empty object that holds the player’s orbit/state vectors. The planetarium computes orbital mechanics in the heliocentric frame; for display, all objects are scaled down and translated into the player’s frame, whose origin is always (0,0,0) in the editor. The scaling keeps objects a reasonable distance apart to avoid rendering artefacts.

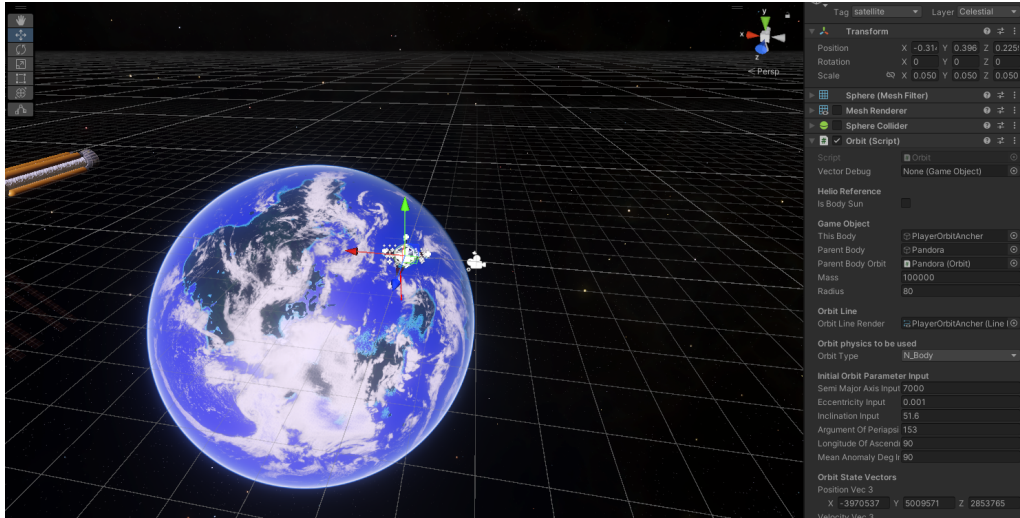


Figure 22. In the editor the player ship sits at (0,0,0) while the planet “orbits” the player. The initial orbit parameters are visible in the inspector.

A function in `Planetarium.cs` runs every update for all tagged celestial or satellite objects and computes the transformation vector.

Listing 14. Computing the floating-origin transform.

```
1 private void getFloatingTransformation()
2 {
3     if (focalPointOrbit)
4         focalPointOrbit = focalPoint.GetComponent<Orbit>();
5
6     floatingOriginTransform = Vector3d.zero - focalPointOrbit.helioCentricPosition;
7     floatingOriginTransformGamePos = Vector3d.toVector3(floatingOriginTransform /
8     planetariumScale);
9     focalPoint.transform.position = Vector3.zero;
10 }
```

The transformation is then applied to every object:

```
1 thisOrbit.gameObject.transform.position =
2     Vector3d.toVector3(thisOrbit.helioCentricPosition / planetariumScale) +
3     floatingOriginTransformGamePos;
```

4 Atmospheric Scattering

Perhaps the strongest visual component of the game is a physically based atmospheric-scattering effect that can be instantiated for each planet. Unity’s default physically based atmosphere exists as a single instance in a skybox and runs only in HDRP, so a custom solution was required.

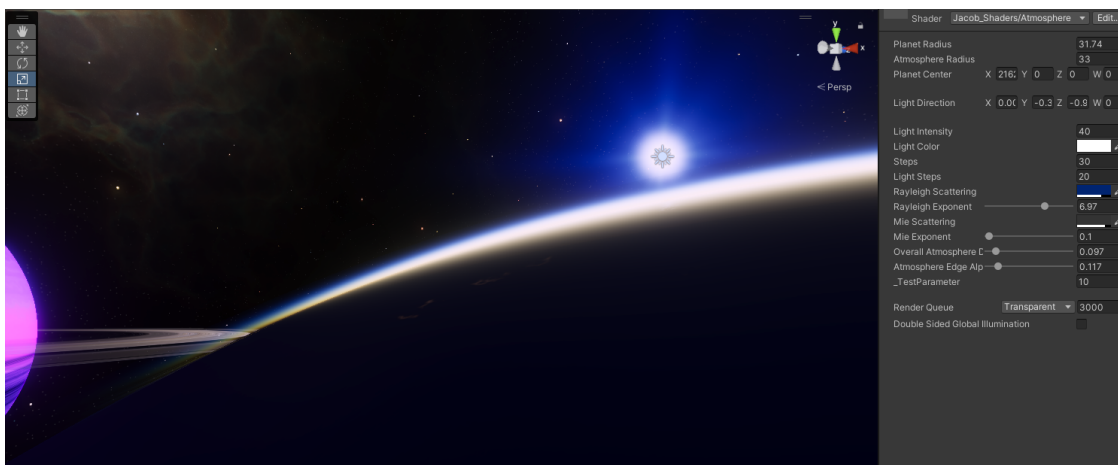


Figure 23. Per-planet physically based atmospheric scattering, with the shader parameters exposed in the inspector.

To create the effect, a mathematical model of how light scatters in an atmosphere — due to gases and particles — must be implemented on the GPU. Scattering is a nonlinear problem that depends on many conditions (optical depth, light source, and so on) and cannot be solved cheaply by approximation, so ray marching is used to solve it iteratively for each pixel. This is computationally expensive, but performance scales well with the GPU’s number of parallel cores. The implementation is based on the treatment in NVIDIA’s *GPU Gems 2*, Chapter 16.²

4.1 Mathematical Model of the Atmosphere

The mathematical model must capture not only the shape of the atmosphere but also the relevant light properties: scattering, the phase function, and the in- and out-scattering functions. All of these are implemented in the fragment shader.

4.1.1 The Shader

A shader is specialised code compiled natively to machine code that runs exclusively on the GPU. If there is a ball in the scene that must be rendered, a shader somewhere tells the GPU *what* to render, *where*, and in what colour. The end result of the atmosphere shader is, for every pixel on screen, a `float4` value to render:

```
1 return float4(color.xyz, _AtmoAlpha * (colorBrightness + _AtmoAlphaCutoff));
```

This `float4` is a single-precision four-component vector whose first three components are the red, green, and blue values and whose last component is the transparency (alpha).

²<https://developer.nvidia.com/gpugems/gpugems2/part-ii-shading-lighting-and-shadows/chapter-16-accurate-atmospheric-scattering>

4.1.2 The Vertex Shader

The vertex shader is the first stage in the rendering pipeline. When an object is placed in the editor, Unity stores the position of each vertex in game space. The vertex positions are passed into the vertex shader, where each triangular face is enclosed by three adjacent vertices.

Listing 15. A minimal vertex (“interpolator”) function.

```

1 Interpolator vert(meshData v)
2 {
3     Interpolator output;
4     output.pos    = UnityObjectToClipPos(v.vertex);
5     output.normal = v.normal;
6     // return output;
7 }
```

The mesh data is passed in as `v`; Unity’s built-in `UnityObjectToClipPos`, the model–view–projection (MVP) matrix, transforms a vertex from game space to clip space.

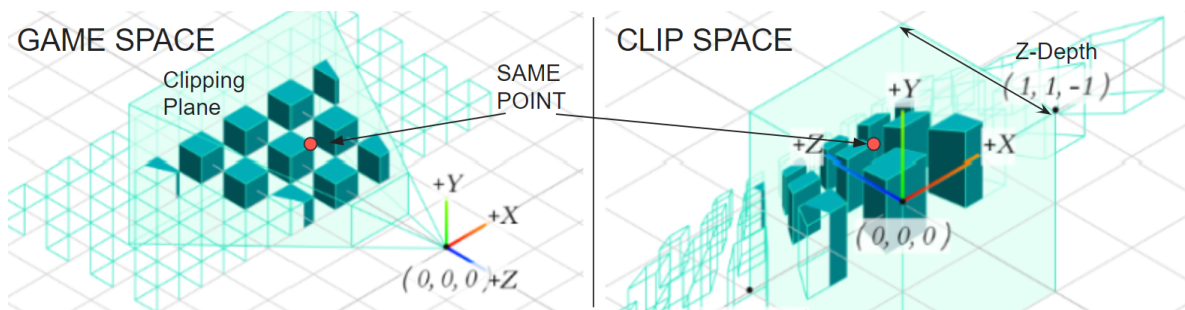


Figure 24. A vertex (red dot) is the same point in game space and in clip space, but occupies different positions in each. Mapping between them is the role of the MVP matrix.

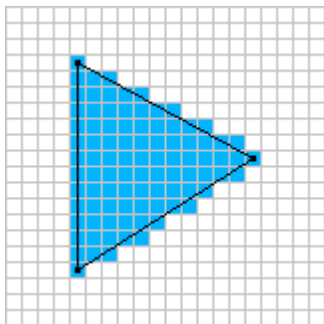
Other quantities are computed too: the UV, the outward-facing normal direction (which tells the GPU which side to render), and the view direction from the camera to the vertex.

```

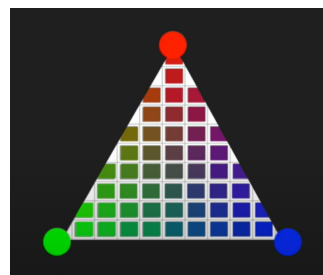
1 output.uv        = v.uv;
2 output.startPos  = mul(unity_ObjectToWorld, v.vertex).xyz;
3 output.viewDir   = normalize(output.startPos - _WorldSpaceCameraPos.xyz);
4 output.worldPos  = mul(unity_ObjectToWorld, float4(v.vertex.xyz, 1)).xyz;
```

4.1.3 Rasterization

Rasterization is automatic. For every set of three adjacent vertex outputs, the three points are assembled into a triangle; the rasterizer then computes which screen pixels the triangle encloses (Figure 25).



(a) Pixels covered by a triangle.



(b) Per-pixel fragments.

Figure 25. The rasterizer determines which pixels a triangle covers; partially covered pixels become fragments shaded by the fragment shader.

4.1.4 The Fragment Shader

The fragment shader runs on the GPU once for every pixel on screen. Zooming in on the pixels, some are only partially enclosed by the triangle; such a pixel is *fragmented*. The fragment shader takes in interpolated data from the rasterized vertices. For example, if the three vertices of a triangle have different colours, the fragment shader averages them by distance and computes the colour each pixel should display, producing a smooth shade.

4.2 Application to Atmospheric Scattering

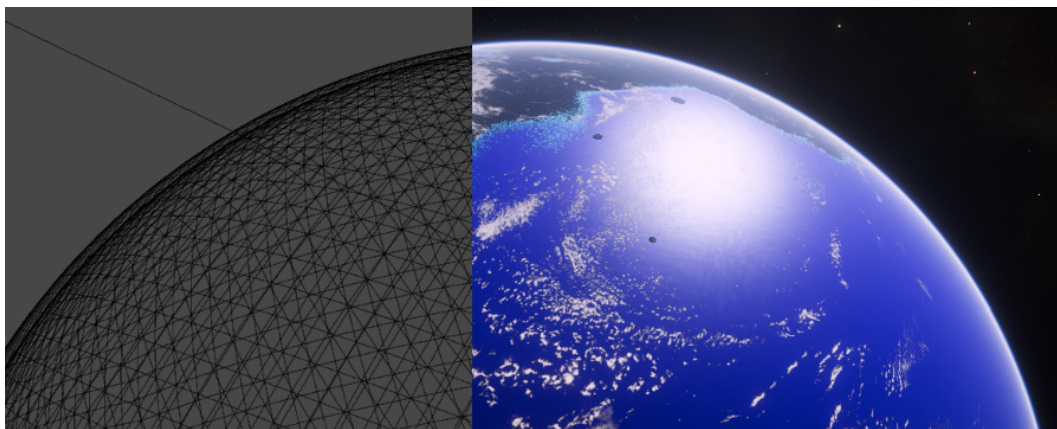


Figure 26. The atmosphere mesh (left) and the rendered result (right). Only view direction and vertex position are needed from the vertex shader; all scattering math runs per pixel in the fragment shader.

For a transparent atmosphere we do not need most of the vertex shader's features (mesh colours or UVs), because no textures or colours are assigned to the mesh. Only the view direction and vertex position are used; all scattering computations run per pixel in the fragment shader. This could be optimised in future by migrating some work to the vertex shader, since objects close to the camera always have fewer vertices than pixels, but that is harder to implement.

4.2.1 Scattering Functions

Two scattering effects occur in reality and must be simulated: **Rayleigh** scattering and **Mie** scattering.



Figure 27. Rayleigh scattering only. Blue light is scattered by small particles such as gas or water molecules, while other wavelengths are absorbed — the familiar blue sky.

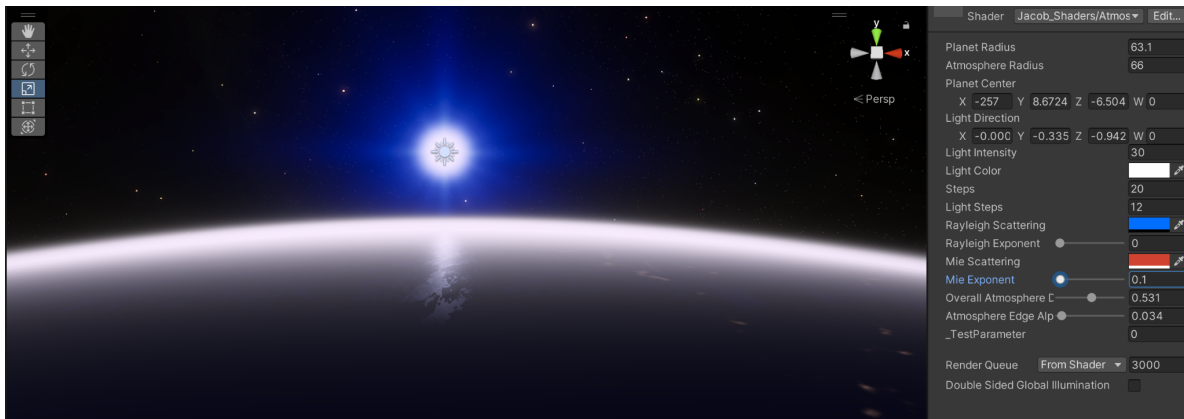


Figure 28. Mie scattering only. Mostly white light is scattered, and the amount depends on the direction of the incoming light. Mie scattering is caused mainly by larger particles such as aerosols in the lower atmosphere.

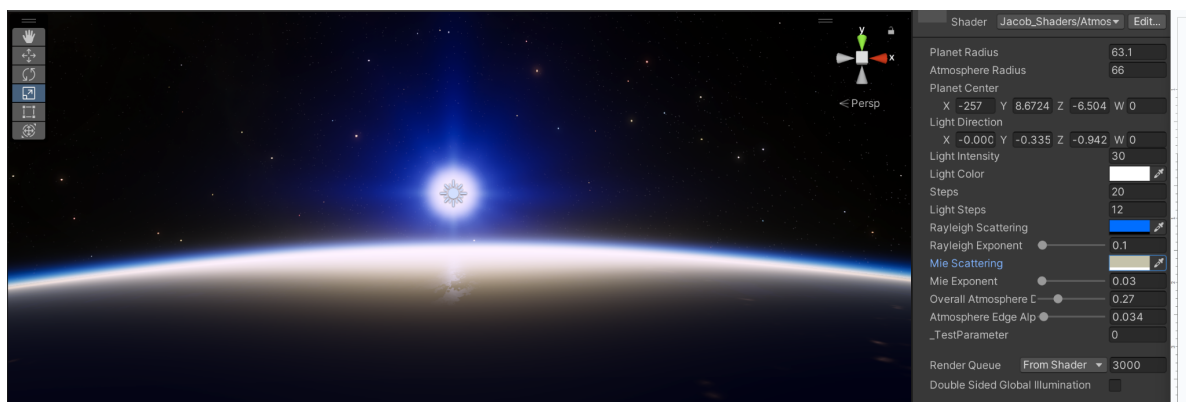


Figure 29. Rayleigh and Mie combined: the upper atmosphere is bluish while the thicker lower atmosphere scatters more white/yellow/red light depending on the camera and Sun positions.

4.3 The Mathematics of Atmospheric Scattering

All of the following runs in the fragment shader.

Step 1 — ray–sphere intersection. The first step is to determine the optical depth of each pixel. Optical depth is the distance a ray travels within the atmosphere — from point A to point B along the view direction. This is computed using trigonometry, given the planet’s centre C (in clip space) and the clip-space position.

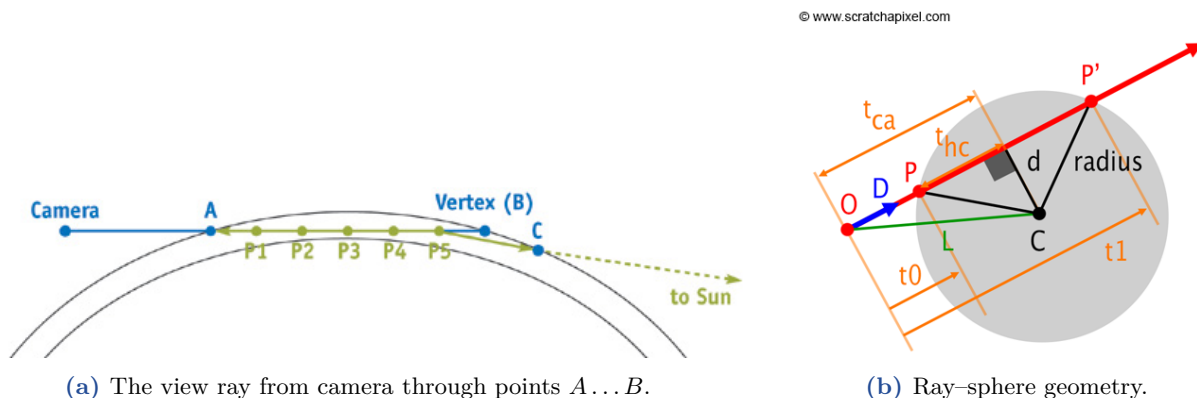


Figure 30. Determining the segment of the view ray that lies inside the atmosphere.

Listing 16. Ray–sphere intersection (returns near/far hits t_0, t_1).

```

1 // Returns sphere intersection for ray direction: t0 = first hit, t1 = second hit
2 bool SphereIntersect(float3 ro, float3 rd, out float t0, out float t1, out bool t0_ground, bool
    isPlanet)
3 {
4     // Dot of planet-to-camera vector with the ray (tangent).
5     float t = dot(_PlanetCenter - ro, rd);
6     float3 pM = ro + rd * t;
7
8     // Ray height above planet origin

```



```

9   float height = sqLength(pM - _PlanetCenter);
10  float atmoRadi_Sqr = _AtmosphereRadius * _AtmosphereRadius;
11
12  // No ray intersection
13  if (height > atmoRadi_Sqr)
14      return false;
15  // ... solve for t0, t1
16  }

```

Step 2 — the phase function. When light hits a particle, some is absorbed and some is scattered, but the amount scattered is not uniform with direction. The *phase function* gives how much light is scattered at each light-step point, given the angle between the incoming light (the Sun) and the view direction. For example, if the camera is directly opposite the Sun, the phase function indicates the scattering is brightest.

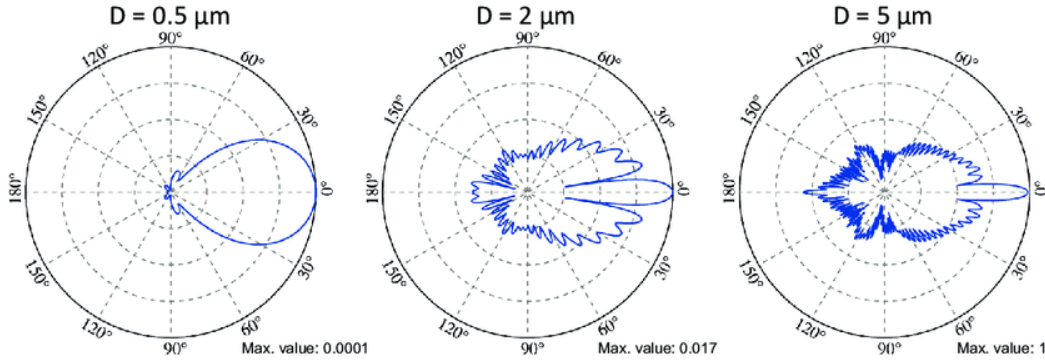


Figure 31. Mie phase functions for increasing particle diameter D . Larger particles scatter increasingly in the forward direction.

The phase function is

$$F(\theta, g) = \frac{3(1 - g^2)}{2(2 + g^2)} \cdot \frac{1 + \cos^2 \theta}{(1 + g^2 - 2g \cos \theta)^{3/2}}, \quad (33)$$

where θ is the angle from a light-step point to the camera, and g is the scattering (asymmetry) coefficient — typically -0.75 to -0.999 for Mie. We use $g = 0$ for Rayleigh, so light simply passes through and becomes blue as an approximation.

Listing 17. Per-pixel Rayleigh and Mie phase values (adapted from Bruneton’s reference).

```

1  float mu = dot(i.viewDir, normalize(-_LightDirection));
2  float g = _MieScattering.y;
3
4  // Phase values for Rayleigh and Mie.
5  float phaseR = 3.0 / (16.0 * PI) * (1 + mu * mu);
6  float phaseM = 3.0 / (8.0 * PI) *
7      ((1.f - g * g) * (1.f + mu * mu)) /
8      ((2.f + g * g) * pow(1.f + g * g - 2.f * g * mu, 1.5f));

```

Step 3 — light march. Given the user-defined number of light steps, we determine where each light-march point lies; this is later used for the out-scattering function. Each point is equally spaced; its position is found by adding a distance vector. For each step, the density is sampled from the height h and a manual density exponent (falloff) H_0 . The optical-depth contribution of a march of length s is

$$\int_{P_a}^{P_b} \exp\left(\frac{-h}{H_0}\right) ds. \quad (34)$$

Listing 18. Light march accumulating Rayleigh/Mie optical depth.

```

1 // Light march steps based on intersection
2 bool LightMarch(float3 p1, float3 rd, float l, out float2 lightDepth)
3 {
4     float ds = 1 / _LightSteps;
5     float time = 0;
6     lightDepth = float2(0, 0);
7
8     for (int i = 0; i < _LightSteps; i++)
9     {
10        float3 p = p1 + rd * (time + ds * 0.5);
11        float height = (length(p - _PlanetCenter) - _PlanetRadius);
12
13        if (height < 0)
14            return false;
15
16        // Optical depth (density exponent term)
17        lightDepth.x += exp(-height / _RayleighExponent) * ds;
18        lightDepth.y += exp(-height / _MieExponent) * ds;
19
20        time += ds;
21    }
22    return true;
23 }
```

Step 4 — in/out scattering. The out-scattering function gives how much light is scattered *away* for each pixel. The integral term is exactly the optical depth computed in the light march:

$$t(P_a, P_b, \lambda) = 4\pi K(\lambda) \int_{P_a}^{P_b} \exp\left(\frac{-h}{H_0}\right) ds. \quad (35)$$

In code, the Rayleigh term (left) and Mie term (right) are combined into τ :

```

1 float3 tau = rsRGB * (opticalDepth.x + opticallightDepth.x)
2             + msRGB * 1.1 * (opticalDepth.y + opticallightDepth.y);
```

The in-scattering (attenuation) function gives the energy *added* due to light from the Sun. The attenuation is the exponential of τ , integrated with respect to the step size ds :

$$I_v(\lambda) = I_s(\lambda) K(\lambda) F(\theta, g) \int_{P_a}^{P_b} \exp\left(\frac{-h}{H_0}\right) \exp(-t(PP_c, \lambda) - t(PP_a, \lambda)) ds. \quad (36)$$

In code we simply exponentiate the out-scattering term to obtain the attenuation:

```

1 float3 attenuation = float3(exp(-tau.x), exp(-tau.y), exp(-tau.z));
2 sumR += attenuation * hr;
3 sumM += attenuation * hm;

```

Step 5 — output. The colour of each pixel due to Rayleigh and Mie is combined into an RGB value and passed to the fragment output. An alpha cutoff based on atmospheric height blends the top of the atmosphere with dark space.

Listing 19. Combining Rayleigh and Mie into the final pixel colour.

```

1 // Combine Rayleigh and Mie
2 float3 color = (sumR * rsRGB * phaseR + sumM * msRGB * phaseM) * _LightIntensity * _LightColor;
3
4 // Alpha dropoff based on ray optical depth, ignoring ground
5 float colorBrightness = (color.r + color.g + color.b) / 3;
6
7 return float4(color.xyz, _AtmoAlpha * (colorBrightness + _AtmoAlphaCutoff));

```

Step 6 — dynamic atmosphere properties (Atmosphere.cs). The ray–sphere intersection uses the planet’s central position as the sphere centre, and the Sun direction changes over time. A companion script keeps these shader properties updated each frame. In play mode the per-instance material reference is used; in builds the shared material reference is used.

Listing 20. Updating shader uniforms for light direction and planet centre.

```

1 if (DirectionalLight != null)
2 {
3     // Set normalized vector of directional light.
4     Vector3 normalDirection = DirectionalLight.transform.forward.normalized;
5
6     if (Application.isPlaying)
7         atmosphereShader.material.SetVector("_LightDirection", normalDirection);
8     else
9         atmosphereShader.sharedMaterial.SetVector("_LightDirection", normalDirection);
10 }
11
12 gameObjectTransform = transform.position;
13 // Set center of sphere as atmosphere sphere position
14 if (Application.isPlaying)
15     atmosphereShader.material.SetVector("_PlanetCenter", gameObjectTransform);
16 else
17     atmosphereShader.sharedMaterial.SetVector("_PlanetCenter", gameObjectTransform);

```

5 Math and Physics: Ship Module

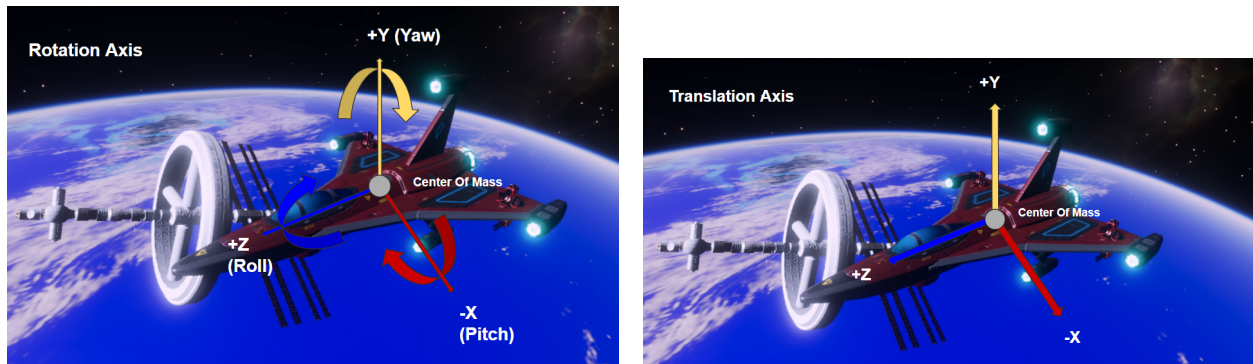
The ship module is responsible for all ships — players and enemies — which are rendered in their own scene, where the player ship sits at the floating origin, again to overcome Unity’s precision issue.

Component	Key function	Source (excl. deps.)
Inertia tensor	Governs how the spacecraft reacts to external forces; applies forces to the spacecraft; handles local movement and rotation.	<code>InertiaTensor.cs</code>
Ship controller	Converts user input into thruster commands.	<code>Spaceship.cs</code> , <code>PIDController.cs</code>
Input system	Handles user input.	<code>ShipInput.cs</code>

Table 3. Components of the ship module.

5.1 Spacecraft Axes

In space any spacecraft has six degrees of freedom across two modes. The rotational mode is similar to an aircraft's (pitch, roll, and yaw). Unlike an aircraft — which uses rotation to control its direction of movement — a spacecraft uses dedicated thrusters to control translation, the second mode.



(a) Rotation axes: pitch ($-X$), yaw ($+Y$), roll ($+Z$).

(b) Translation axes about the centre of mass.

Figure 32. The six degrees of freedom, expressed in Unity's (Z, Y, X) axis convention.

5.2 Reaction Control

In space there is no air or reaction mass to act upon, so a spacecraft cannot use the control surfaces of an aircraft. Instead it uses a reaction-control method: by Newton's third law, every action has an equal and opposite reaction, so shooting puffs of gas at high speed produces an equal and opposite reaction on the craft. Control of the spacecraft across all six axes is achieved this way.

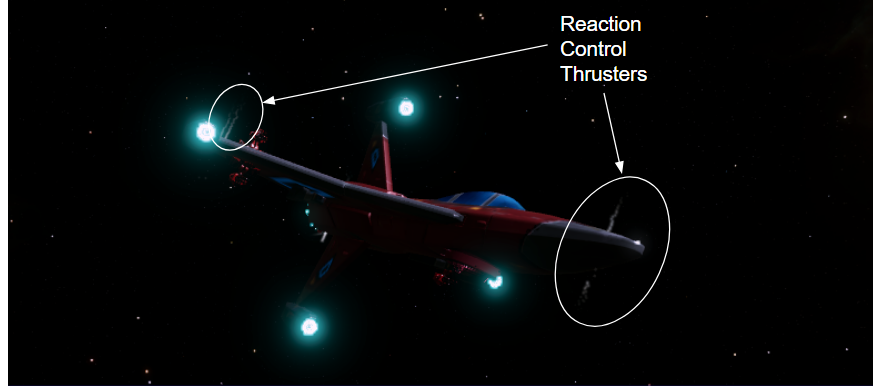


Figure 33. Reaction-control thrusters distributed around the hull.

To determine how much a spacecraft moves when a reaction force is applied, we first compute its inertia. There are two modes: *rotational* and *translational*. Rotational inertia is the resistance to rotation; it is proportional to the size of the craft and to its mass distribution (a long craft has higher inertia about its long axes). Translational inertia is the resistance to linear motion; for this the craft can be treated as a point mass at its centre of mass.

5.3 Inertia Tensor: Rotation

To compute the resultant rate of rotation efficiently, we implement a rank-2 inertia tensor. Consider first the moment of inertia of a simple cube about its centre of mass, along each of the three axes:

$$I = \frac{1}{6} m a^2, \quad (37)$$

where I is the moment of inertia (kg m^2), m the mass of the cube, and a the cube's side length.

To approximate the composite inertia of an irregular shape such as a spacecraft, we imagine filling its volume with cubes and summing their moments of inertia.

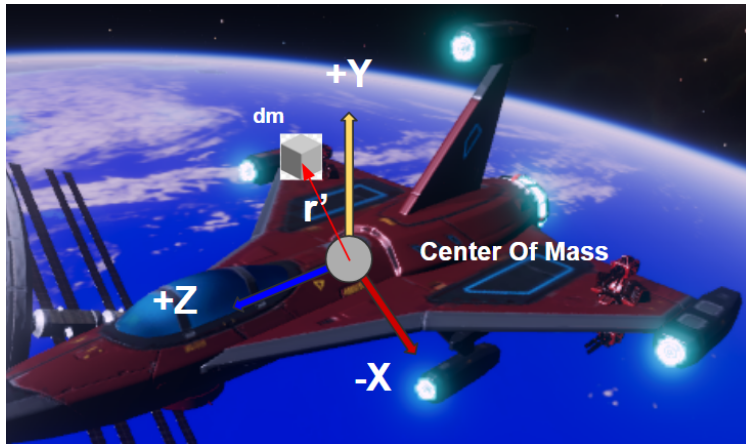


Figure 34. A module's mass element dm at offset $\mathbf{r}'$ from the composite centre of mass.

For each point mass (box), the angular momentum about the centre of mass G is

$$\mathbf{H}_G = \int_m \mathbf{r}' \times (\boldsymbol{\omega} \times \mathbf{r}') dm, \quad (38)$$

where $\mathbf{H}_G$ is the angular momentum about G , $\boldsymbol{\omega}$ the angular velocity, $\mathbf{r}'$ the distance from the cube centre to the spacecraft's centre of mass, and dm the cube's mass. Expanding for each of the three axes,

$$\begin{aligned}\mathbf{H}_G = & \left(\omega_x \int_m (x'^2 + y'^2 + z'^2) dm - \int_m (\omega_x x' + \omega_y y' + \omega_z z') x' dm \right) \mathbf{i} \\ & + \left(\omega_y \int_m (x'^2 + y'^2 + z'^2) dm - \int_m (\omega_x x' + \omega_y y' + \omega_z z') y' dm \right) \mathbf{j} \\ & + \left(\omega_z \int_m (x'^2 + y'^2 + z'^2) dm - \int_m (\omega_x x' + \omega_y y' + \omega_z z') z' dm \right) \mathbf{k},\end{aligned}\quad (39)$$

where x', y', z' are the components of $\mathbf{r}'$ and $\mathbf{i}, \mathbf{j}, \mathbf{k}$ are unit direction vectors.

The moments of inertia about the x , y , and z axes are

$$I_{xx} = \int_m (y'^2 + z'^2) dm, \quad I_{yy} = \int_m (x'^2 + z'^2) dm, \quad I_{zz} = \int_m (x'^2 + y'^2) dm, \quad (40)$$

and the products of inertia, which account for how mass is distributed about two axes simultaneously, are

$$I_{xy} = I_{yx} = \int_m x' y' dm, \quad I_{xz} = I_{zx} = \int_m x' z' dm, \quad I_{yz} = I_{zy} = \int_m y' z' dm. \quad (41)$$

Substituting into the expansion gives

$$\begin{aligned}\mathbf{H}_G = & (I_{xx}\omega_x - I_{xy}\omega_y - I_{xz}\omega_z) \mathbf{i} \\ & + (-I_{yx}\omega_x + I_{yy}\omega_y - I_{yz}\omega_z) \mathbf{j} \\ & + (-I_{zx}\omega_x - I_{zy}\omega_y + I_{zz}\omega_z) \mathbf{k}.\end{aligned}\quad (42)$$

Extracting the angular velocity, this is written in tensor form:

$$\begin{pmatrix} H_{Gx} \\ H_{Gy} \\ H_{Gz} \end{pmatrix} = \begin{pmatrix} I_{xx} & -I_{xy} & -I_{xz} \\ -I_{yx} & I_{yy} & -I_{yz} \\ -I_{zx} & -I_{zy} & I_{zz} \end{pmatrix} \begin{pmatrix} \omega_x \\ \omega_y \\ \omega_z \end{pmatrix}. \quad (43)$$

For our application in Unity we simplify by neglecting the off-diagonal (product) terms and using only the principal-axis inertia tensor. This is a good trade-off between realism and a faster development cycle, because it lets us integrate with Unity's rigid-body system.

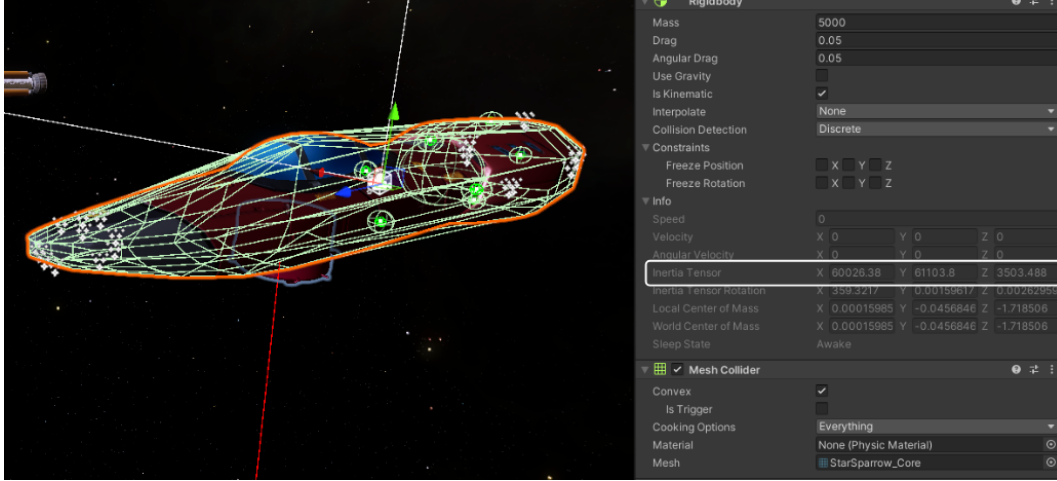


Figure 35. Unity’s rigid body computes a per-collider inertia tensor from the convex mesh. The engine, however, exposes no way to combine several rigid bodies into a composite tensor — which is why the system is reproduced externally.

In Unity, attaching a rigid body to an object lets the engine use the collider and its mass to compute the inertia tensor; the convex mesh approximates the object’s true shape. The engine breaks the mesh into primitive shapes and integrates the principal- axis theorem under the hood to obtain a composite tensor — but it does not expose this code. If we want a spacecraft made of separate modular parts, each its own rigid body, there is no built-in way to combine them into a composite inertia tensor and centre of mass. Hence we reproduce this system ourselves.

5.4 Composite Centre of Mass

The first step is the spacecraft’s composite centre of mass, which depends on how mass is distributed across the modules. Each module has its own mass and centre of mass; the composite centre of mass about each axis is

$$x_G = \frac{\sum_i m_i \bar{x}_i}{m_{\text{total}}}, \quad y_G = \frac{\sum_i m_i \bar{y}_i}{m_{\text{total}}}, \quad z_G = \frac{\sum_i m_i \bar{z}_i}{m_{\text{total}}}, \quad (44)$$

where $\bar{x}_i, \bar{y}_i, \bar{z}_i$ are the module centre-of-mass coordinates relative to a common reference (the geometric centre of the command module, i.e. `transform.position`) and m_i is the module mass.

Using a recursive search we find all child modules attached to the command module, and by summing all module centres of mass we obtain the composite centre of mass.

Listing 21. Mass-weighted accumulation of the composite centre of mass.

```

1 for (int i = 0; i < VehicleModules.Count; i++)
2 {
3     compositeCenterOfMass += VehicleModules[i].worldCenterOfMass * VehicleModules[i].mass;
4     compositeMass          += VehicleModules[i].mass;
5 }
6
7 compositeCenterOfMass = compositeCenterOfMass / compositeMass;
```

5.5 Composite Inertia Tensor

To obtain the composite inertia tensor, we first find the individual inertia tensor of each module, then use the parallel-axis theorem to sum them. The tensor generalisation of the parallel-axis theorem states that if the inertia tensor is known about a centre of mass, the inertia about a new arbitrary point (here the ship's composite centre of mass) is

$$\mathbf{J} = \mathbf{I} + m[(\mathbf{R} \cdot \mathbf{R}) \mathbf{E}_3 - \mathbf{R} \otimes \mathbf{R}], \quad (45)$$

where $\mathbf{J}$ is the inertia tensor about the new point, $\mathbf{I}$ the module's tensor about its own centre of mass, $\mathbf{R}$ the offset from the module's centre of mass to the ship's composite centre of mass, $\mathbf{E}_3$ the identity matrix, $\otimes$ the outer product, and m the module mass.

Unity stores the inertia tensor as a vector representing the diagonal inertia matrix, so we convert it to a 3×3 matrix — embedded in a 4×4 because Unity only provides `Matrix4x4` — by adding a zero column. The entries m_{00}, m_{11}, m_{22} are the diagonal components.

Listing 22. Converting Unity's diagonal-vector tensor to a matrix.

```
1 private Matrix4x4 convertTensorTo4x4(Vector3 tensor)
2 {
3     Matrix4x4 result = new Matrix4x4();
4     result.m00 = tensor.x;
5     result.m11 = tensor.y;
6     result.m22 = tensor.z;
7     return result;
8 }
```

Unity's `Matrix4x4` supports only matrix–vector operations; matrix–matrix addition, multiplication, and inversion are not provided, so these were implemented manually.

Listing 23. Matrix–matrix addition, subtraction, and multiplication.

```
1 private Matrix4x4 matrixMatrixOperation(string sign, Matrix4x4 matrix1, Matrix4x4 matrix2)
2 {
3     Matrix4x4 resultMatrix = Matrix4x4.zero;
4     if (sign != "+" && sign != "-" && sign != "*")
5     {
6         Debug.Log("Error matrixMatrixOperation input correct sign.");
7         return resultMatrix;
8     }
9
10    for (int i = 0; i < 16; i++)
11    {
12        if (sign == "+") resultMatrix[i] = matrix1[i] + matrix2[i];
13        if (sign == "-") resultMatrix[i] = matrix1[i] - matrix2[i];
14    }
15
16    if (sign == "*")
17    {
18        int rA = 4, cA = 4, cB = 4;
19        float temp = 0;
20        for (int i = 0; i < rA; i++)
21            for (int j = 0; j < cB; j++)
22            {
23                temp = 0;
```



```

24         for (int k = 0; k < cA; k++)
25             temp += matrix1[i, k] * matrix2[k, j];
26         resultMatrix[i, j] = temp;
27     }
28 }
29 return resultMatrix;
30 }

```

Listing 24. Outer product of a vector (used in the parallel-axis term).

```

1 private Matrix4x4 OuterProduct(Vector3 vector)
2 {
3     Matrix4x4 outerProductMatrix = new Matrix4x4();
4     outerProductMatrix.m00 = vector.x * vector.x;
5     outerProductMatrix.m01 = vector.x * vector.y;
6     outerProductMatrix.m02 = vector.x * vector.z;
7
8     outerProductMatrix.m10 = vector.y * vector.x;
9     outerProductMatrix.m11 = vector.y * vector.y;
10    outerProductMatrix.m12 = vector.y * vector.z;
11
12    outerProductMatrix.m20 = vector.z * vector.x;
13    outerProductMatrix.m21 = vector.z * vector.y;
14    outerProductMatrix.m22 = vector.z * vector.z;
15
16    return outerProductMatrix;
17 }

```

A companion `getInverse` routine implements the standard 4×4 cofactor/adjugate inverse (determinant followed by the transposed cofactor matrix divided by the determinant). Iterating over every attached module then yields the composite inertia tensor. Note that for the offset vector we transform the module's world centre of mass, and the ship's world centre of mass, into the parent object's local frame.

Listing 25. Accumulating the composite inertia tensor via the parallel-axis theorem.

```

1 Matrix4x4 _identity = Matrix4x4.identity;
2 _identity.m33 = 0;
3
4 foreach (Rigidbody module in VehicleModules)
5 {
6     Vector3 offset = parentGameObject.transform.InverseTransformDirection(
7         module.worldCenterOfMass - COM_Combined_World);
8
9     Matrix4x4 offsetOuterProd = OuterProduct(offset);
10    float offsetDot = Vector3.Dot(offset, offset);
11    offsetOuterProd.m33 = 0;
12
13    Matrix4x4 moduleInertiaTensor = convertTensorTo4x4(module.inertiaTensor);
14    Matrix4x4 translation = matrixMatrixOperation("-",
15        matrix4x4MultiplyScalar(offsetDot, _identity), offsetOuterProd);
16    Matrix4x4 translatedTensor = matrixMatrixOperation("+",
17        moduleInertiaTensor, matrix4x4MultiplyScalar(module.mass, translation));
18
19    translatedTensor.m01 = 0; translatedTensor.m02 = 0;
20    translatedTensor.m10 = 0; translatedTensor.m12 = 0;

```

```

21 translatedTensor.m20 = 0; translatedTensor.m21 = 0;
22
23 output = matrixMatrixOperation("+", output, translatedTensor);
24 }

```

5.6 Torque

Torque is given by

$$\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F}, \quad (46)$$

where $\boldsymbol{\tau}$ is the torque vector relative to the command module's local axes, $\mathbf{r}$ the displacement relative to the ship's composite centre of mass, and $\mathbf{F}$ the negative of the thrust (the reaction force on the craft is opposite the thrust direction).

```

1 Vector3 distance      = thruster.getLocalPosition(parentGameObject.transform, COM_Combined_World
  );
2 Vector3 thrustingDir = thruster.getThrustDirection(parentGameObject.transform);
3 Vector3 torque       = Vector3.Cross(Vector3.Normalize(thrustingDir) * -thisThrust, distance);

```

To find how that torque changes the craft's angular velocity, we reuse the angular-momentum relation, replacing angular momentum with torque and multiplying by the inverse of the inertia tensor to obtain angular acceleration $\boldsymbol{\alpha}$ (the change in angular velocity, $\Delta\boldsymbol{\omega}$, caused by a force over time):

$$\begin{pmatrix} \alpha_x \\ \alpha_y \\ \alpha_z \end{pmatrix} = \begin{pmatrix} I_{xx} & -I_{xy} & -I_{xz} \\ -I_{yx} & I_{yy} & -I_{yz} \\ -I_{zx} & -I_{zy} & I_{zz} \end{pmatrix}^{-1} \begin{pmatrix} \tau_x \\ \tau_y \\ \tau_z \end{pmatrix}. \quad (47)$$

```

1 Vector3 deltaOmega = VehicleInertiaTensorMatrixInverse.MultiplyVector(torque);

```

The $\Delta\boldsymbol{\omega}$ value, in the local player-ship frame, then decides which thrusters must fire for each rotational axis. The largest component of $\Delta\boldsymbol{\omega}$ means the force is most efficiently translated to that axis, which determines the thruster mapping for each axis.

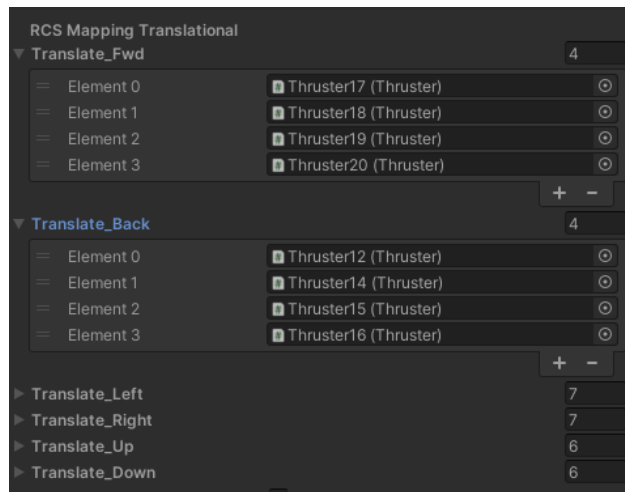


Figure 36. Rotational RCS thruster mapping, grouped by axis (pitch, roll, yaw).

5.7 Translational Movement

Treating the assembled ship as a point mass, translational movement is the sum of all forces via $\mathbf{F} = m\mathbf{a}$:

$$\mathbf{a} = \frac{\sum_i \mathbf{F}_i}{m_{\text{composite}}}. \quad (48)$$

```

1 Vector3 thrustingDir      = thruster.getThrustDirection(parentGameObject.transform);
2 Vector3 thrustingForce    = Vector3.Normalize(thrustingDir) * -thisThrust;
3 Vector3 translationalAccel = thrustingForce / compositeMass;
4 Vector3 worldTranslationalAccel = parentGameObject.transform.TransformDirection(
    translationalAccel);

```

The same computation determines which thruster is most effective for each translational axis, which is used to update the translational RCS mapping.

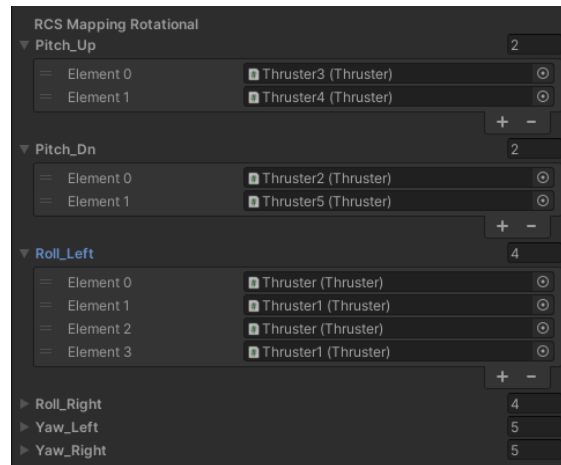


Figure 37. Translational RCS thruster mapping (forward/back/left/right/up/down).

The inertia-tensor script implements an *add-force-at-position* function, similar to the rigid body's built-in method, but based on the composite inertia tensor and the composite centre of mass. Both rotational and translational accelerations are computed for each time step.

Listing 26. Applying a thruster force at a position to the composite body.

```

1 public void addForceAtPos(Thruster thruster, float thisThrust, float dt)
2 {
3     // Rotational
4     Vector3 angularVelocityDelta = parentGameObject.transform.TransformDirection(
5         getAngularVelocityChange(thruster, -thisThrust));
6     AngularVelocityDelta = angularVelocityDelta;
7
8     AngularVelocity.x += (AngularVelocityDelta.x * dt);
9     AngularVelocity.y += (AngularVelocityDelta.y * dt);
10    AngularVelocity.z += (AngularVelocityDelta.z * dt);
11
12    // Translational
13    Vector3 thrustingDir      = thruster.getThrustDirection(parentGameObject.transform);
14    Vector3 thrustingForce    = Vector3.Normalize(thrustingDir) * -thisThrust;

```

```

15  Vector3 translationalAccel = thrustingForce / compositeMass;
16
17  Vector3 worldTranslationalAccel = parentGameObject.transform.TransformDirection(
    translationalAccel);
18  worldAccelLoop += worldTranslationalAccel;
19
20  worldVelocity += (worldTranslationalAccel * dt);
21  localVelocity = parentGameObject.transform.InverseTransformDirection(worldVelocity);
22  }

```

5.8 Handling Position and Rotation of the Game Object

All motion of the game object is handled by the inertia-tensor script. World position and world velocity are stored in double precision and run continuously in each fixed update. When another function calls add-force-at-position, the angular and linear accelerations are updated; each fixed update may receive many such calls, which simply accumulate. The acceleration is then integrated into world position and world velocity.

This world position and velocity is always the floating-origin position, whose initial value is zero. The player's displacement from the origin is computed and applied to all other ship objects in the scene to transform them into the player's frame, keeping the player ship at the game origin. The player's position and velocity in the planetarium are stored in the `Orbit.cs` heliocentric position and velocity.

Listing 27. Per-fixed-update integration of the floating-origin body.

```

1  public void integrate(float _dt)
2  {
3      if (initState)
4      {
5          worldVelocityPrev = worldVelocity;
6
7          // Integrate angular acceleration
8          Quaternion rotation = Quaternion.Euler(-AngularVelocity);
9
10         // Integrate translational movement
11         worldPosition = worldPosition + (Vector3d.toVector3d(worldVelocity) * _dt);
12
13         if (isPlayerShip)
14         {
15             // Update floating origin
16             _floatingOrigin.worldPosPlayer = worldPosition;
17             _floatingOrigin.updatePosition();
18             COM_Combined_World = Vector3.zero;
19         }
20         else
21         {
22             COM_Combined_World = parentRigidBody.worldCenterOfMass;
23         }
24
25         // Rotate the object about the composite COM
26         RotateAround(parentGameObject.transform, COM_Combined_World, rotation);
27
28         worldAccelVec = worldAccelLoop;
29         localAccelVec = parentGameObject.transform.InverseTransformDirection(worldAccelLoop);

```

```

30     worldAccelLoop = Vector3.zero;
31
32     if (COM_Indicator != null)
33         COM_Indicator.transform.position = COM_Combined_World;
34 }
35 }

```

6 Crafting Dynamic Spaceships and Paths

This section covers the modular construction system and the pathfinding logic, which together provide the dynamic ship-customisation experience and the navigational intelligence of NPCs.

6.1 Modular Ship Construction

6.1.1 The Goal

The modular system is inspired by the game *Captain Forever*, whose premise is that the player builds spaceships *while flying them*: blast an enemy ship apart, grab a piece with the mouse, and bolt it to your own ship. Because everything is physically simulated, ship design has a large impact on flight characteristics and combat capability. *Endangered Orbits* pursues the same goal, but where *Captain Forever* is 2D, *Endangered Orbits* is 3D — which introduced new problems to solve.

6.1.2 Defining Modules

To create the basic building blocks of the ship — the modules — we use an interface hierarchy built on inheritance. The root interface is `IModule`, which serves as the foundation for all module types and ensures every module exposes a uniform set of properties and behaviours, creating a consistent, extensible framework.

Listing 28. The root module interface.

```

1 public interface IModule
2 {
3     public GameObject GetGameObject { get; }
4     public string Name { get; set; }
5     public int Health { get; set; }
6     public int MaxHealth { get; set; }
7     public int Mass { get; set; }
8     public DamageResistancePercent DamageResistance { get; set; }
9     public List<IAttachable> AttachedModules { get; set; }
10 }

```

Because an interface cannot itself express implementation, a traditional class hierarchy realises these interfaces, and the game modules inherit from those classes — primarily to avoid duplicating code. Most properties (such as `Health`) are self-explanatory; two warrant more detail:

- **AttachedModules** is the list of modules attached to this module. Each module may have many attachment points; this list references any modules occupying those points, implicitly

forming a tree where each node has a number of children equal to its attachment points. The list is primarily a convenience; all child modules are also set as Unity children of the parent game object, so the hierarchy is likewise reachable via `transform`.

- **DamageResistance** is an instance of `DamageResistancePercent`, a class that stores a float clamped between 0 and 1 — thus a percentage.

The second interface, `IAttachable`, inherits `IModule` and enforces the functionality that lets a module attach to another. The only additional state it requires is a flag denoting whether the module is currently attached.

Listing 29. The attachable-module interface.

```

1 public interface IAttachable : IModule
2 {
3     public bool IsAttached { get; }
4     public void Attach(AttachmentPoint attachmentPoint);
5     public void Eject();
6     public void Disconnect(bool removeFromList = true);
7     public void PickUp();
8     public void Release(Vector3 releaseVelocity);
9     public AttachmentPoint FindThisConnectionPoint();
10 }
```

The purpose of each function:

1. **Attach** joins two modules: it aligns the current module's game object with the target attachment point; sets the current module as a child of the parent module's game object (retrieved from the target attachment point); changes the current module's rigid body to kinematic; and sets the attachment flag to true.
2. **Eject** launches the module away from what it is connected to: it unsets the parent-child relationship; changes the rigid body back to dynamic; sets the attachment flag to false; applies a force to push the module away from the parent; and finally takes the player ship's current velocity and slowly applies it to the module so the module halts relative to the player ship. This last step prevents modules from flying off into space so that the player can still retrieve them. `Eject` is never invoked directly by the player; it triggers when a module can no longer validly connect to its parent — for example, if the parent module is destroyed in combat.
3. **Disconnect** simply severs the connection between two modules: it unsets the parent-child relationship, disables rigid-body interactions, and sets the attachment flag to false. Disabling rigid-body interactions ensures the disconnected module — while constrained to the mouse cursor — does not collide with other objects, so the player cannot use it to interact physically with the environment.
4. **PickUp** makes the changes needed (such as disabling the rigid body) to prepare a module to be constrained to the player's mouse.
5. **Release** reverts the module to its original state after being picked up and applies a velocity proportional to the mouse movement, up to a maximum.

6. **FindThisConnectionPoint** returns the first attachment point on the current module whose `connectionPoint` flag is set to true.

When two modules join, they stay together because the child game object is a child of the parent; setting the rigid body to kinematic keeps the child in the position it is supposed to be.

6.1.3 Attachment Points

Attachment points are game objects positioned where a module can connect to others. They are controlled by the `AttachmentPoint` class, which manages two things: how to attach other modules at the point (based on the game object's position and rotation), and enabling/disabling the mesh renderer at appropriate times (including changing the material at runtime).

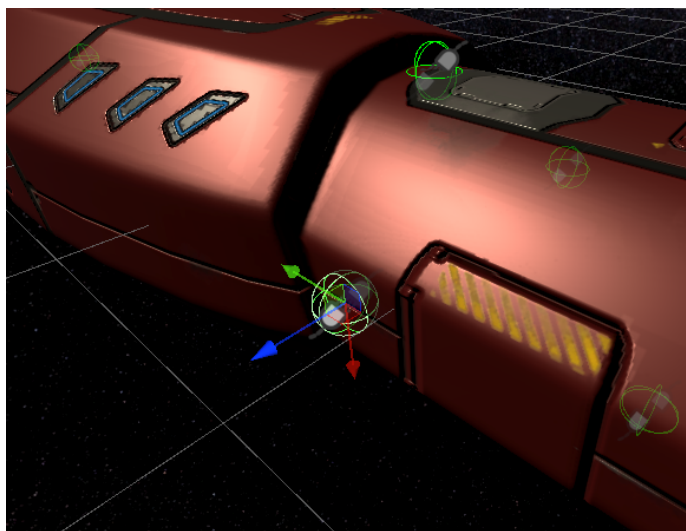


Figure 38. An attachment point in the editor; note the forward vector.

When `Attach` is called, it uses an `AttachmentPoint` to determine how the other module should align and connect. The process aligns the forward vectors of the connecting points using quaternion rotations, ensuring the module is oriented correctly relative to the target. It also adjusts the Z-axis rotation to match the attachment point, computes the translation to position the module exactly at the point, and sets the module's game object as a child of the attachment point's parent so they move as one object. Finally, the attachment points on both modules are marked as occupied and their visual meshes are hidden, indicating a successful connection.

To determine which attachment point to use on the module the player is holding, each point carries a boolean `connectionPoint`. When attachment is invoked, a linear search over the module's children returns the first point with this flag set to true — which is then joined as described.



(a) Holding: points highlight green.

(b) Hovering: point turns pink.

(c) Attached: points hidden.

Figure 39. The three visual states of a module during construction.

6.1.4 Control

In *Captain Forever*'s 2D world, holding a module poses no spatial problems. In the 3D space of *Endangered Orbits*, we must now reason about how far the mouse is from the camera, occlusion of attachment points, and the difficulty of flying a ship with six degrees of freedom while building it. Some compromises were necessary; assembling modules is not as streamlined as in *Captain Forever*.

The first problem is where to float a module while the player holds it. The most user-friendly solution is to hold the module the same distance from the camera as the ship core. To achieve this, a script attached to the main camera creates a plane, the `InteractionPlane`: the camera's forward vector aligns the plane, and the plane's centre is placed at the ship core via its transform. When the loop that updates a held module runs, it finds the point on the plane to reposition the module to, using `ScreenPointToRay` on the main camera. This finds the distance of the plane from the camera and allows accurate positioning of the module.

While the player holds a module, an `if` clause in the interaction controller's `Update` calls the function that updates the held module's position, and also checks whether the player is hovering over an attachment point. By placing the attachment points in their own layer, a layer mask isolates them in a ray cast. When the player hovers over a point, the module is positioned over the prospective attachment area — with an offset so it appears to hover over the point. The occlusion problem is handled simply by selecting the first attachment point the ray cast hits.

Finally, flying the ship while building it in a full Newtonian simulation is really only solvable through player skill (and would likely benefit from an additional controller such as a joystick, since the mouse is otherwise overloaded). We settled on three modes: a *build* mode that allows adding and removing modules; a *camera* mode, identical to build mode except the ship cannot be modified; and a *combat* mode, in which the ship orients toward where the player is looking and lateral-movement controls are activated.

6.2 Pathfinding

In *Endangered Orbits*, players move through an environment quite unlike typical games: there is no traditional ground to walk on, which makes standard pathfinding systems — like Unity's — unsuitable. With no ground, there is no surface from which to build a map of walkable areas; the concept of a nav mesh does not apply in space.

One possible solution is an *octree*, a data structure that efficiently represents 3D space and could support algorithms such as Dijkstra or A*. We did not use this method: Unity’s built-in pathfinder does not support octrees directly, so adopting one would require developing our own pathfinder, which would be overkill here. To quote *WALL-E*: “There’s a lot of space in space.” It is possible to handle 99% of situations with a far more rudimentary algorithm.

6.2.1 It’s Raycasts All the Way Down

Since obstacles are both rare and convex in most situations, no advanced pathfinder is needed. We shoot a ray cast at the destination; if no obstacle is found, we simply go. If there is an obstacle, we use its bounding box to plot an intermediate point to fly to first.

The process begins by sizing a sphere from the object’s speed and a base diameter; this sphere represents the “safety” space the vessel wants to keep clear. The script then sphere-casts from the current position toward the destination, checking for obstacles along the way. If the sphere cast detects no obstacles — or an obstacle is near enough to the destination that it would not be an obstacle — a direct path is set. Otherwise the script navigates around the obstacle.

First, the original destination is stored as the final destination; then the obstacle’s size and position are analysed via its bounding box. To find the most efficient detour, we take the smallest dimension of the bounding box (to the side, up, or forward relative to the object’s orientation). Once the shortest direction is determined, a point is set on that side of the object, offset by roughly the sphere diameter computed earlier. This point becomes the new movement destination — an intermediate step toward the final one.

Each intermediate position undergoes the same verification as the original destination. Because of this, a step counter is necessary, checked against a maximum number of pathfinding steps to avoid an infinite loop if the path remains blocked. If the count exceeds the limit, the script falls back to an alternative method (see Section 7).

6.2.2 Object Avoidance

Alongside pathfinding, NPC movement uses a series of simple object-avoidance algorithms (here “our” refers to an instance of an NPC), each run once per update:

1. Maintain a safe buffer in the immediate vicinity, avoiding nearby objects.
2. Monitor objects in a particular layer and avoid them if a collision seems likely.
3. Use the current velocity to project whether we will collide with something, and attempt to avoid it.

Close-proximity check. We run an overlap sphere at the current position and, after filtering ourselves out, inspect the remaining objects. For each, we compute its velocity relative to us; using our velocity vector and the relative velocity vector we obtain an angle. If the angle is $\geq 90^\circ$, the object is moving parallel to or away from us and can be ignored. If it is approaching, we divide the angle by 90 and invert it to obtain a value in $[0, 1]$ representing how directly it comes at us. We then divide the magnitude of that object’s vector by the size of the overlap sphere and invert it,

obtaining another value in $[0, 1]$ representing proximity. Assuming a collision course, we compute a time to impact; if it is more than 10 seconds we ignore the object this cycle. Otherwise we fly away by requesting a velocity that avoids the collision, scaled by the angle and distance factors so we respond more strongly to closer, more direct threats. The requested velocity is stored in a class-context variable to be acted upon in the next `FixedUpdate`.

Layer check. The layer check is similar but came about because firing sphere casts in all directions left too many gaps in vision. Rather than scan a full 360° sphere, we directly check only the things that can move (never more than about ten of them). All such objects in the specified layer are collected into a list and iterated; the angle and velocity checks are much the same. The key difference is that instead of fleeing directly, we introduce a lateral move to dodge: because acceleration is limited, an object closing from a kilometre away moves far faster than we can match. To generate the lateral move, we take the vector to the object, copy it, swap the sign of one component, and take the cross product of the original and the modified copy. This yields a perpendicular vector, in which we request maximum acceleration; the request is stored in the same context variable.

Forward projection (“don’t run into stuff”). We take our current vector and plot where we would be in 10 seconds at constant velocity, then sphere-cast to that position. If we find anything, we compute the nearest point of impact from the hit point and surface normal, enlarge it by half our sphere diameter to keep a safe distance, and compute a perpendicular vector as before. This time we add a search: to find the emptiest direction, we rotate the calculated vector in 10° increments. For each generated perpendicular vector we assess a point in that direction at the projected 10-second travel distance, and test for obstacles there with a small-radius overlap test, recording the shortest distance to any obstacle surface. That distance indicates how safe the direction is; the vector with the greatest clearance is the safest. To evade, we compute the necessary acceleration from the distance to the nearest obstacle boundary and our current speed, and set the evasion velocity as that acceleration times the safest perpendicular vector. This vector is added to the same context variable as the other two steps.

6.2.3 Movement

All movement — for both avoidance and pathfinding — is performed in `FixedUpdate`. This is why every step pools the velocities it computes into a single accumulated movement direction. In the avoidance functions, velocities that have no hope of being attained are often requested; because the magnitude is based on need, when the pooled vector is normalised only the most important action survives. This also prevents the steps from fighting each other and creating chaotic motion. The actual movement is delegated to a movement controller, decoupling the calculations from the method so they work regardless of whether we use the humble `addForce` or a more complicated system. Because all vectors are pooled into a single request, the avoidance systems can freely interrupt pathing as needed and resume once the threat has passed — with no explicit effort to make it so.

The other task completed in `FixedUpdate` is managing the life cycle of the pooled vector via a set of counters that track aspects of movement and evasion. For instance, an `evadeResetUpdateCounter` increments whenever the NPC has not detected anything, indicating a period of non-evasive movement; an `evadingCounter` increases whenever there is a need to evade. The former determines

when the pooled velocity should be cleared so the ship eventually stops; the latter informs the close-evade function — which normally flies directly away from the other object, but once this count grows large enough switches to a perpendicular vector like the other functions.

7 Future Work and Roadmap

Several systems are designed but not yet complete, or are implemented at a basic level with a clear path to improvement. They are collected here.

Spacecraft input and control documentation

The detailed write-up of the `ShipInput` and spacecraft-control layer (the six closed-loop PID controllers introduced in Section 5) is still to be written; the underlying systems exist in the project but are not yet documented here.

Adaptive Runge–Kutta order

The planetarium currently uses RK4 for all integration steps. A natural improvement is to select the integrator order dynamically from an energy-conservation tolerance (for example $\varepsilon \approx 10^{-8}$), promoting to the RK8 scheme of Section 3 only when larger time steps demand it.

State-vector-to-elements code path

The reverse conversion from state vectors to Keplerian elements is derived in full, but parts of the implementation are still to be updated in code to match the derivation.

Atmosphere shader optimisation

All scattering math currently runs per pixel in the fragment shader. Migrating part of the computation to the vertex shader would improve performance for objects close to the camera (which always have fewer vertices than pixels), at the cost of greater implementation complexity.

Tangent Bug pathfinding fallback

When the basic ray-cast/sphere-cast pathfinder of Section 6 fails, it currently hits a placeholder and gives up. The intended fallback is the *Tangent Bug* algorithm, which balances local and global path planning: it detects the nearest obstacle, computes the “tangent” points on the obstacle’s boundary that are visible from the current position, and treats them as candidate waypoints. The algorithm evaluates these tangents to pick the one offering the most efficient route to the goal while minimising the detour, moves to it, then reassesses the environment — repeating from tangent to tangent until it reaches the destination or determines that no viable path exists. Unlike simple ray casting, which only checks for direct obstructions, Tangent Bug considers the shape and orientation of obstacles. It was deprioritised because the game’s environments make it largely a novelty that would rarely be needed.